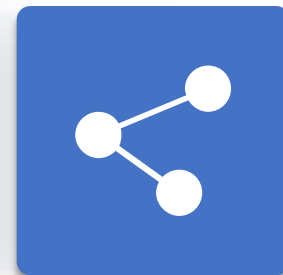




# 人工智能引论

## 第4章 复杂环境中的搜索

授课教师：李静瑶





# 录

- 1 局部搜索
- 2 使用非确定动作的搜索
- 3 部分可观测的搜索
- 4 在线搜索



## 系统化搜索的全局搜索

- 经典搜索按照某种策略系统化地搜索状态空间
- 在搜索过程中，保留了当前路径中的每个节点的选择和相关信息
- 搜索完毕，当前路径就是问题的一个解
- 但在很多问题中，解仅仅是一个状态而不是到达该状态的路径  
例：八皇后问题，重要的是8个皇后在棋盘上的格局，而不是皇后的放置顺序



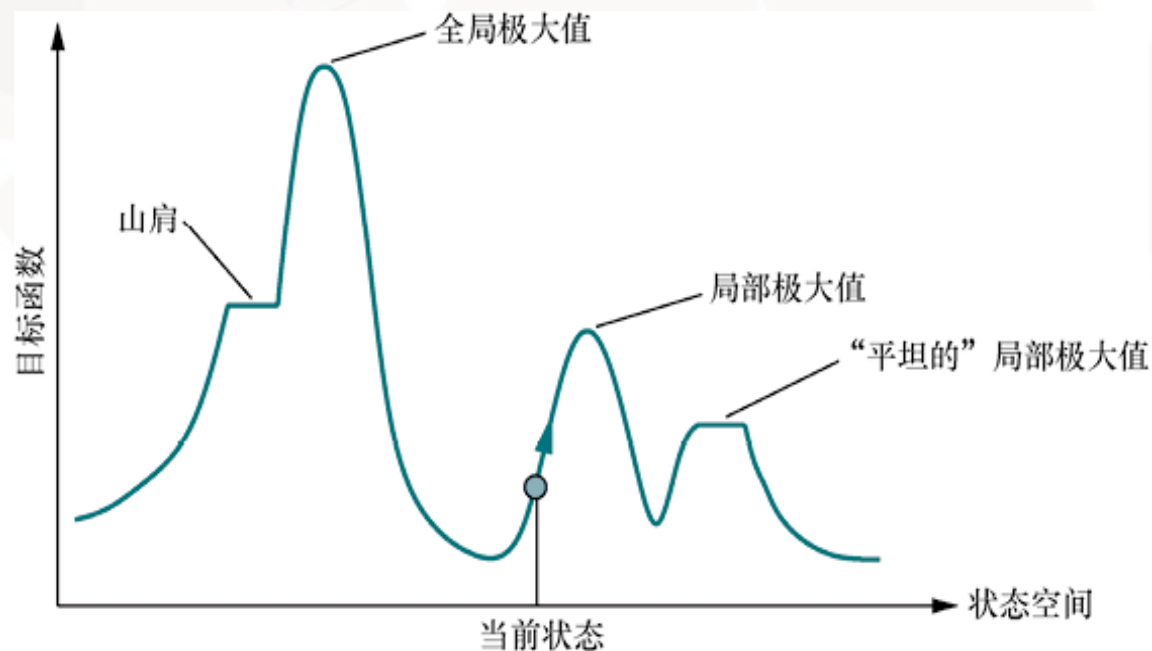
## 非系统化搜索的局部搜索

- 非系统化搜索，可能永远不会探索问题的解实际所在的那部分搜索空间
- 局部搜索从当前状态出发，只能转移到它的邻近状态，不记录搜索路径，也不记录已达状态集 (reached)
- 优点：
  - 在系统化搜索不适用的大规模或无限状态空间中找到合理的解
  - 空间复杂度较低



## 局部搜索

- 也称最优化问题，根据目标函数找到最优状态
- 爬山：目标函数越大越好，找到全局极大值
- 梯度下降：代价函数越小越好，找到全局极小值

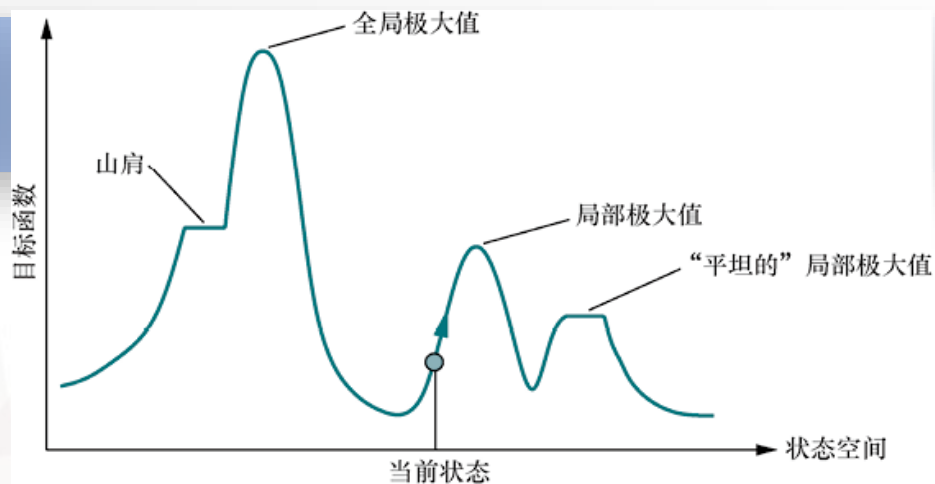


一维状态空间地形图



## 爬山搜索（贪心局部搜索）

- 找到全局极大值
- 只选择当前状态的**最佳**邻近状态，而不考虑全局情况
- 不需要维护搜索树，当前节点只需记录当前状态和目标函数值



**function** HILL-CLIMBING(*problem*) **returns** 一个位于局部极大值的状态

*current*  $\leftarrow$  *problem*.INITIAL

**while** true **do**

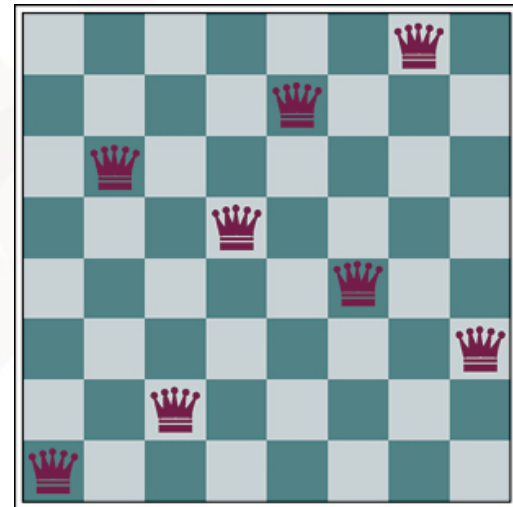
*neighbor*  $\leftarrow$  *current*的值最大的后继状态

**if** VALUE(*neighbor*)  $\leq$  VALUE(*current*) **then return** *current*

*current*  $\leftarrow$  *neighbor*

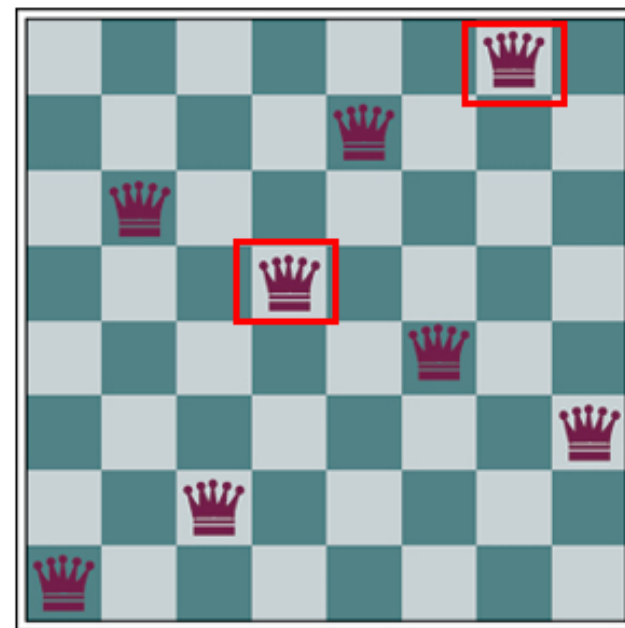
## 例. 八皇后问题

- 状态：棋盘上的一个棋局，表示棋盘上放置了8个皇后，且每列仅1个皇后
- 动作：某个皇后移动到该列的其它7个位置之一
- 每个状态有  $8*7=56$  个后继状态
- 启发式函数 $h$ ：互相攻击的皇后对数
- 求全局极小值，0表示没有互相攻击的皇后，找到解



## 例. 八皇后问题

- 图(a)的状态 $h=1$ , 对应了一个局部极小值
- 当前状态非常接近于一个解, 第4列和第7列的两个皇后会沿对角线互相攻击



(a)





## 例. 八皇后问题

- 一个8皇后状态，其启发式代价估计值 $h = 17$
- 棋盘显示了通过在同一列移动皇后而获得的每一个可能后继的 $h$
- 有8个移动并列最优，其 $h = 12$ 。爬山法将选择它们中的一个

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 18 | 12 | 14 | 13 | 13 | 12 | 14 | 14 |
| 14 | 16 | 13 | 15 | 12 | 14 | 12 | 16 |
| 14 | 12 | 18 | 13 | 15 | 12 | 14 | 14 |
| 15 | 14 | 14 | ♙  | 13 | 16 | 13 | 16 |
| ♙  | 14 | 17 | 15 | ♙  | 14 | 16 | 16 |
| 17 | ♙  | 16 | 18 | 15 | ♙  | 15 | ♙  |
| 18 | 14 | ♙  | 15 | 15 | 14 | ♙  | 16 |
| 14 | 14 | 13 | 17 | 12 | 14 | 12 | 18 |

(b)

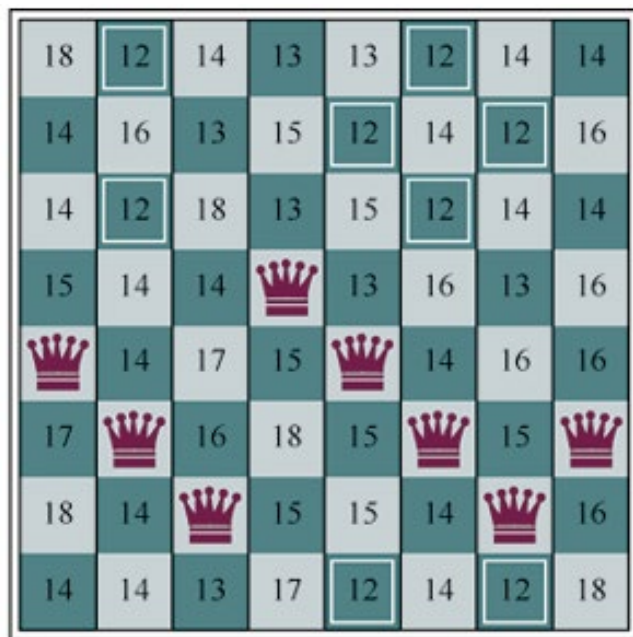


## 例. 八皇后问题

- 如果多个邻近状态 $h$ 值均为最小值，随机选择一个邻近状态进行扩展
- 爬山法也称贪婪局部搜索，只选择当前状态的最佳邻近状态，而不考虑全局情况

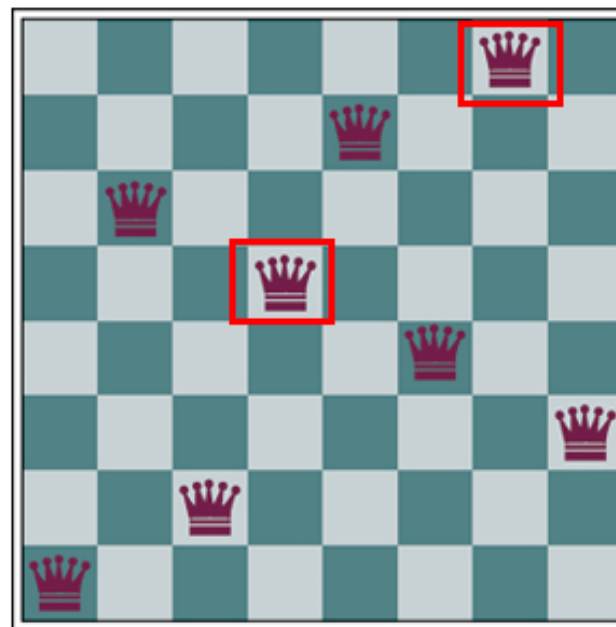
|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 18 | 12 | 14 | 13 | 13 | 12 | 14 | 14 |
| 14 | 16 | 13 | 15 | 12 | 14 | 12 | 16 |
| 14 | 12 | 18 | 13 | 15 | 12 | 14 | 14 |
| 15 | 14 | 14 | ♙  | 13 | 16 | 13 | 16 |
| ♙  | 14 | 17 | 15 | ♙  | 14 | 16 | 16 |
| 17 | ♙  | 16 | 18 | 15 | ♙  | 15 | ♙  |
| 18 | 14 | ♙  | 15 | 15 | 14 | ♙  | 16 |
| 14 | 14 | 13 | 17 | 12 | 14 | 12 | 18 |

(b)



(b)

h=17



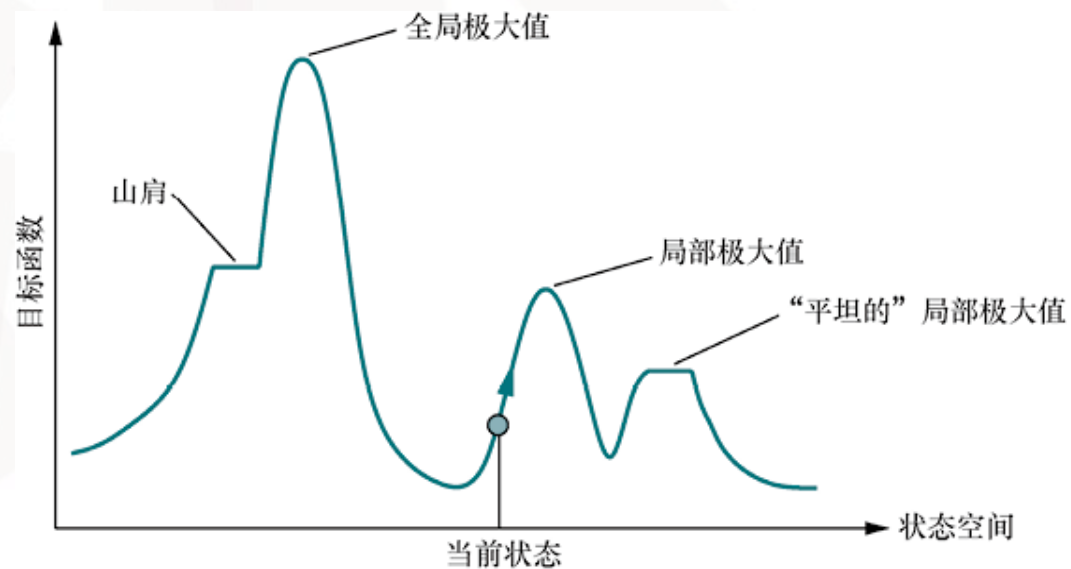
(a)

h=1 (局部极小)



## 爬山搜索陷入困境

- 平台区：
  - 平坦的局部极大值：不存在上坡的出口
  - 山肩：存在上坡的出口
- 局部极大值：比每个相邻状态高但比全局极大值低的峰顶
- **允许横向移动**：最佳邻近节点与当前节点的目标函数值相同（限制连续横向移动的次数）



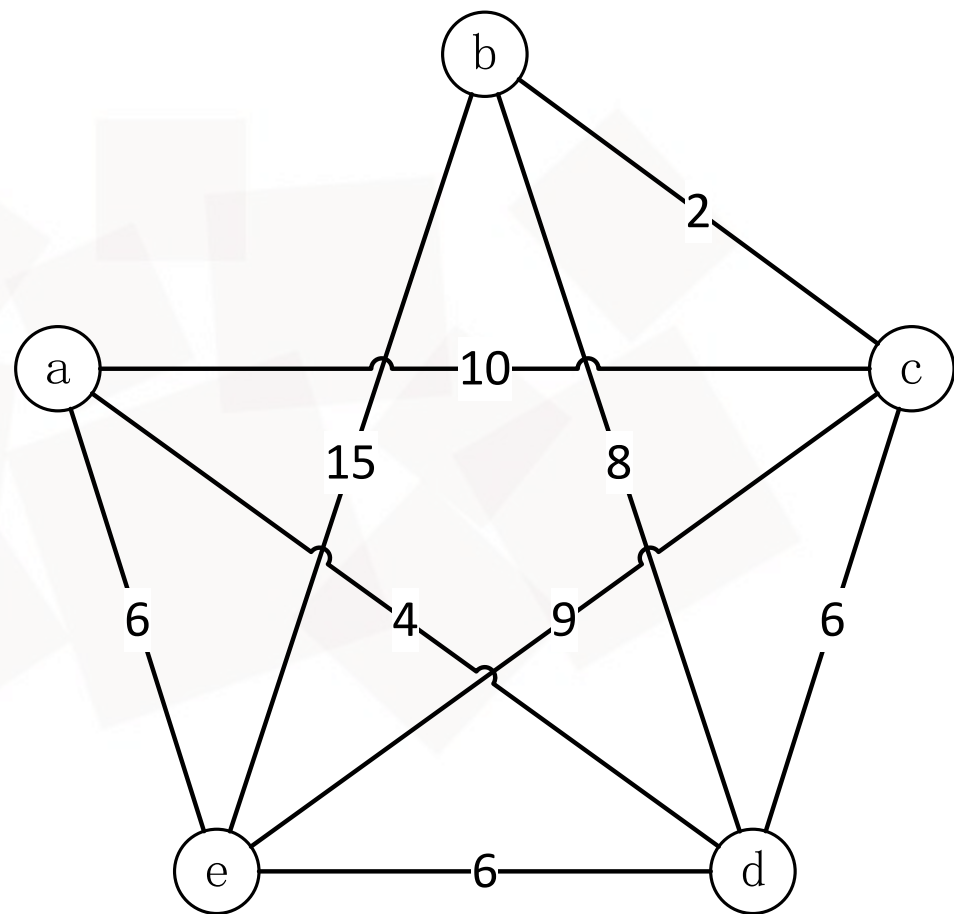


## 爬山搜索的变体

- 随机爬山法：随机选择邻近状态，被选中概率与上坡陡峭程度有关
- 首选爬山法：不断随机地生成后继状态，直到生成一个比当前状态更好的后继状态为止
- 随机重启爬山法：随机生成初始状态，执行一系列爬山搜索，反复进行，直到目标被找到或搜索次数达到一定限制

## 例：旅行商问题

- 一个旅行商需要遍历所有城市，找到所有遍历的最短路径（回路）
- 旅行商经过每个城市恰好一次，相当于所有城市的一个序列（路径）
- 求解方案：基本爬山法  
随机爬山法







状态 $x$ : 当前路径

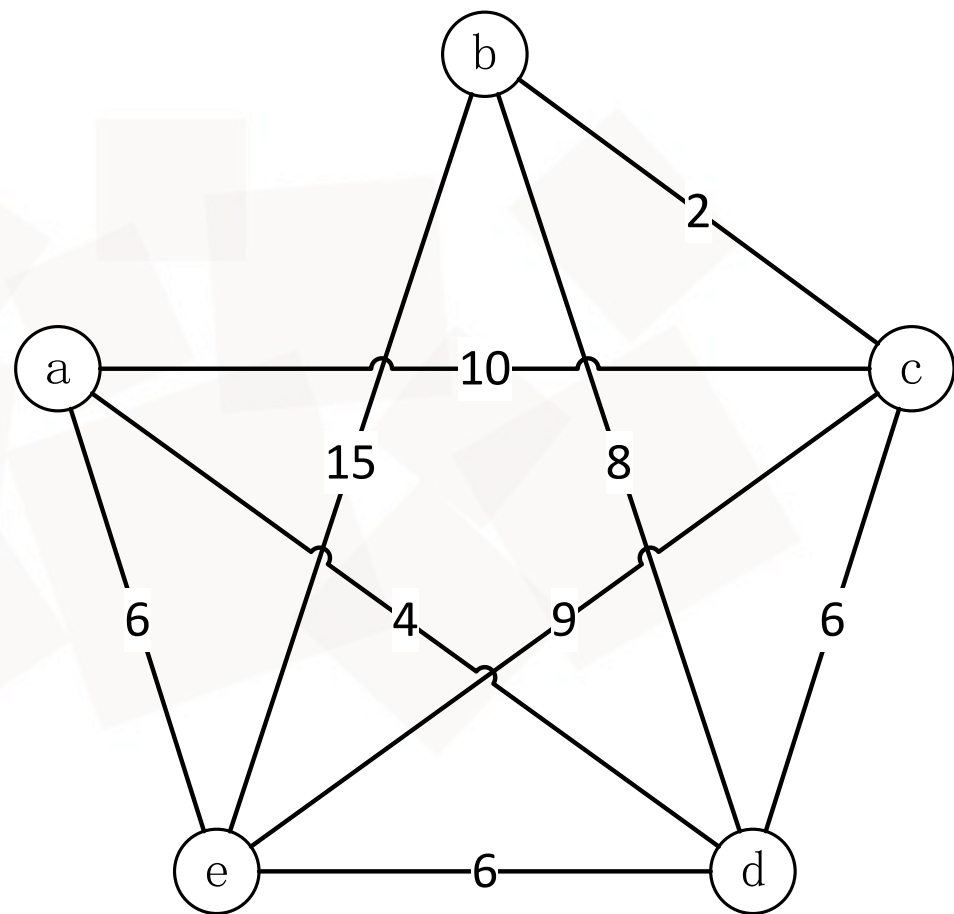
- 初始状态 $x_0$ : 随机选择的任意路径
- 邻近状态 $x_n$ : 当前路径选择任意两个城市交换访问次序得到的新路径
- 邻域 $P$ : 当前路径的邻近状态集

评估函数 $h(x)$ : 当前路径的距离

- 若两城市无路径, 设其距离为 $+\infty$

状态之间的比较方式

- 比较当前路径和任一新路径的距离



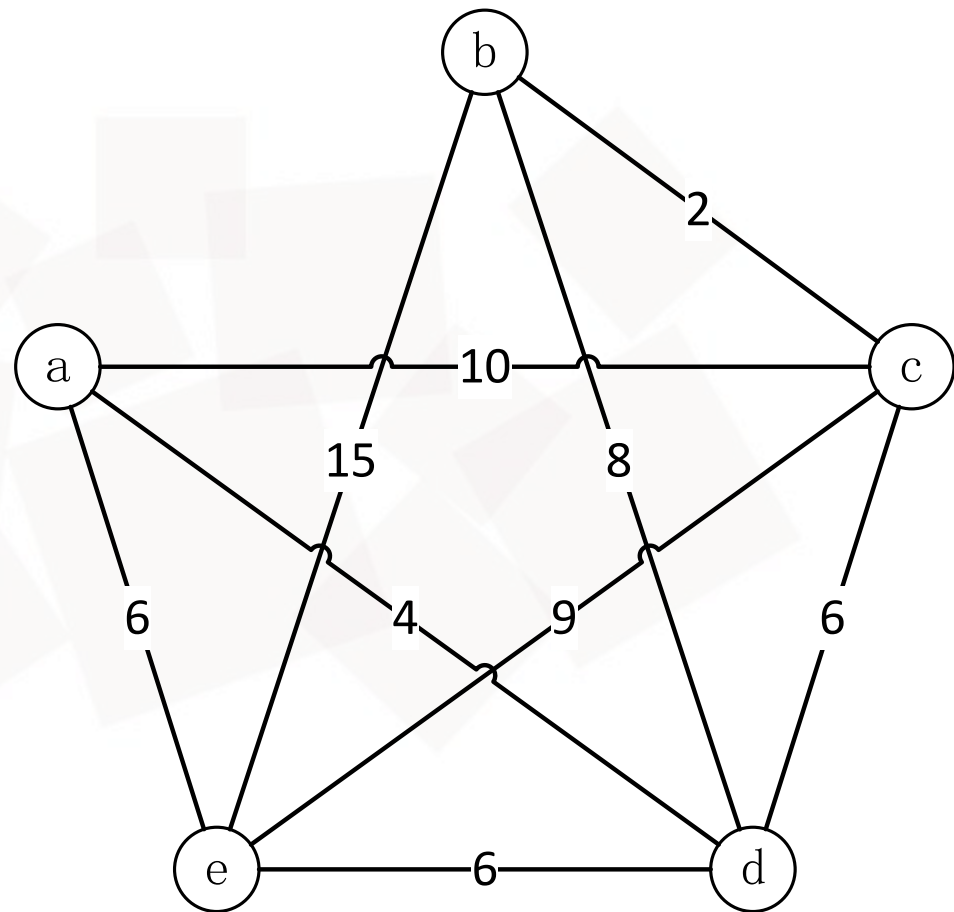
## 基本爬山法

假设从城市a出发，用城市的序列表示该问题的一个可能解

- 初始状态: 随机选择初始可能解

$$x_0 = abcdea$$

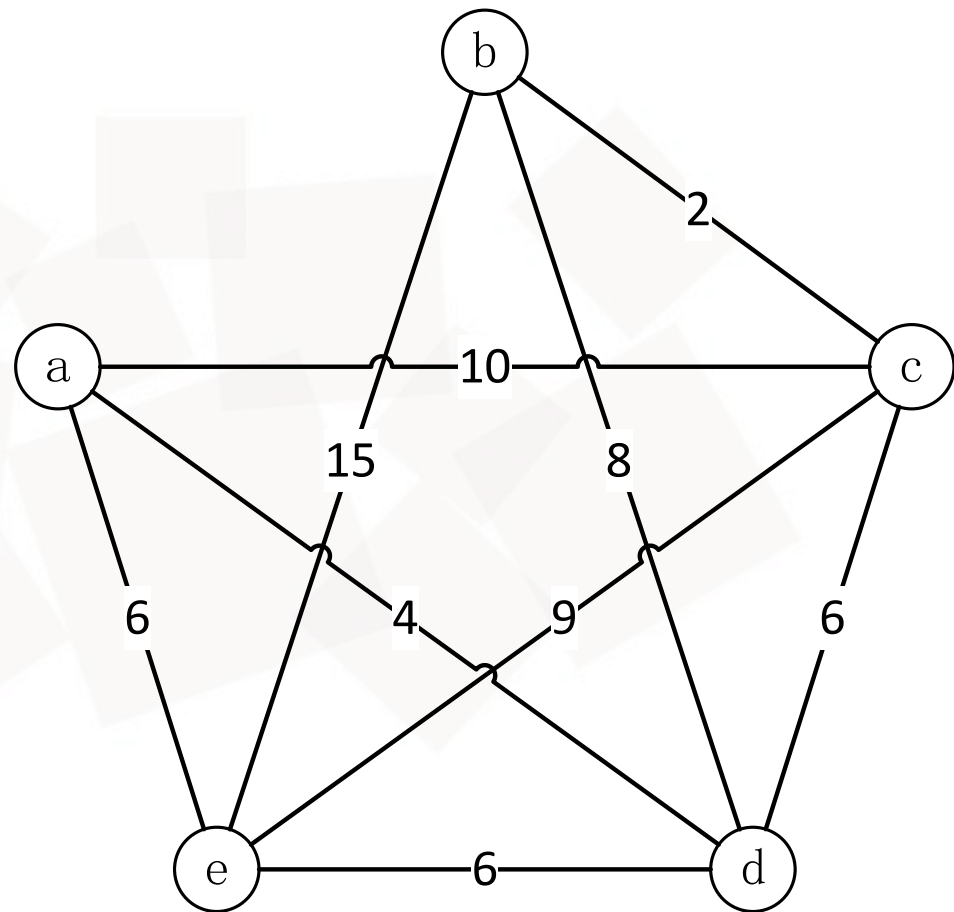
$$f(x_0) = +\infty$$





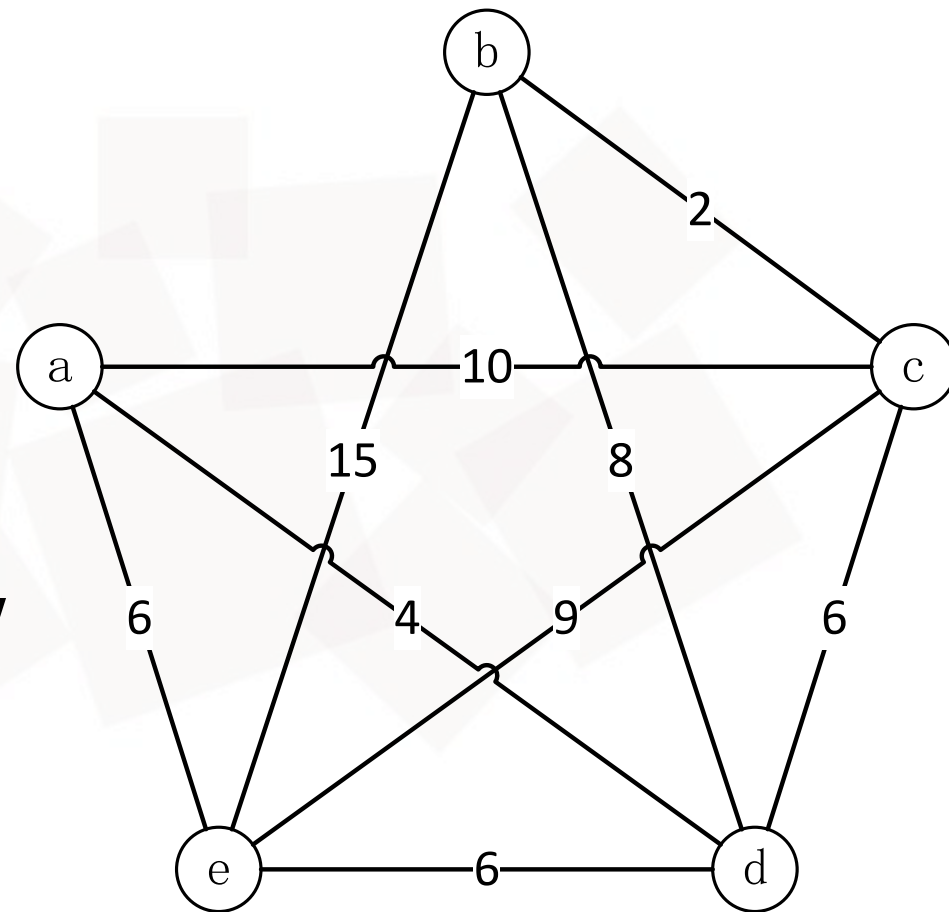
## 第1次搜索 (基本爬山法)

- $x_0 = \text{abcdea}$  的邻域  
 $P = \{\text{acbdea}, \text{adcbea}, \text{aecdba}, \text{abdcea}, \text{abedca}, \text{abceda}\}$
- 从  $P$  中选择  $f$  值最小 的元素  
 $x_n = \text{acbdea}, f(x_n) = 32 < f(x_0) = +\infty$
- 更新  $x_0 = \text{acbdea}$



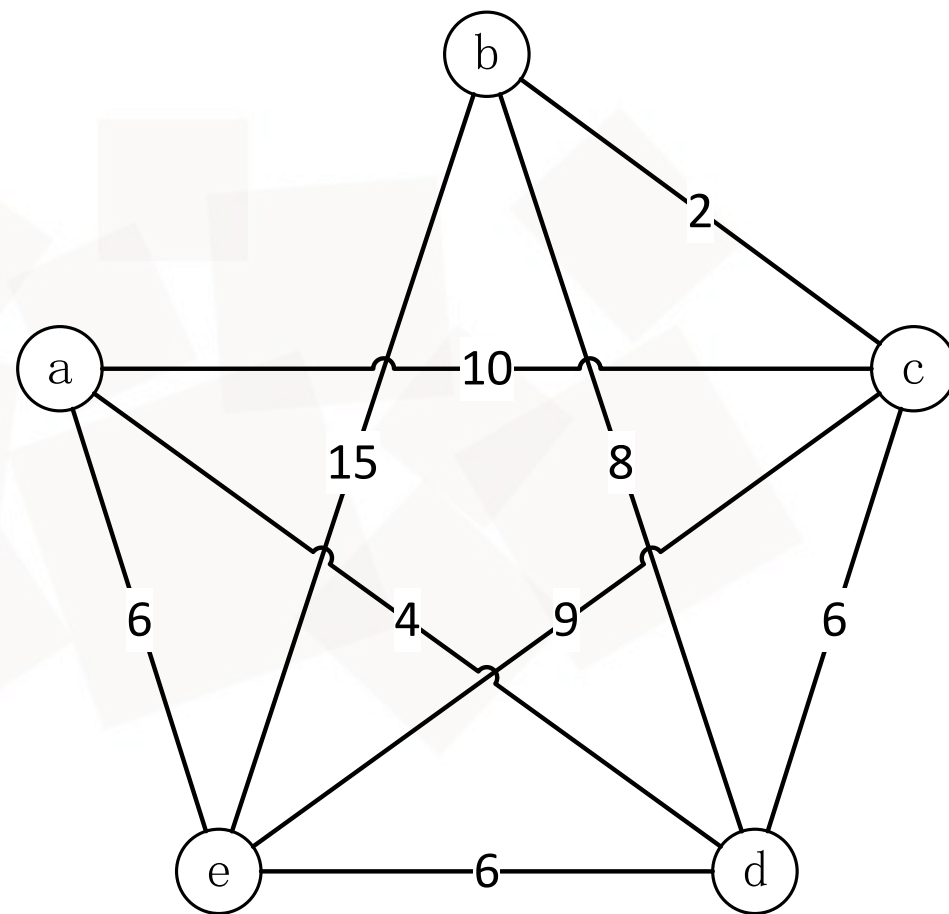
## 第2次搜索 (基本爬山法)

- $x_0 = acbdea$  的邻域  
 $P = \{abcdea, adbcea, aebdca, acdbea, acedba, acbeda\}$
- 从P中选择f值最小的元素  $x_n = adbcea$ ,  
 $f(x_n) = 29 < f(x_0) = 32$
- 更新  $x_0 = adbcea$



## 第3次搜索 (基本爬山法)

- $x_0 = \text{adbcea}$  的邻域  
 $P = \{\text{abddea}, \text{acbdea}, \text{aebcda}, \text{adcbea}, \text{adecba}, \text{adbeca}\}$
- 对  $P$  中所有元素  $x_n$ ,  
 $f(x_n) > f(x_0) = 29$  算法结束
- 搜索结果  
 $x_0 = \text{adbcea}, f(x_0) = 29$





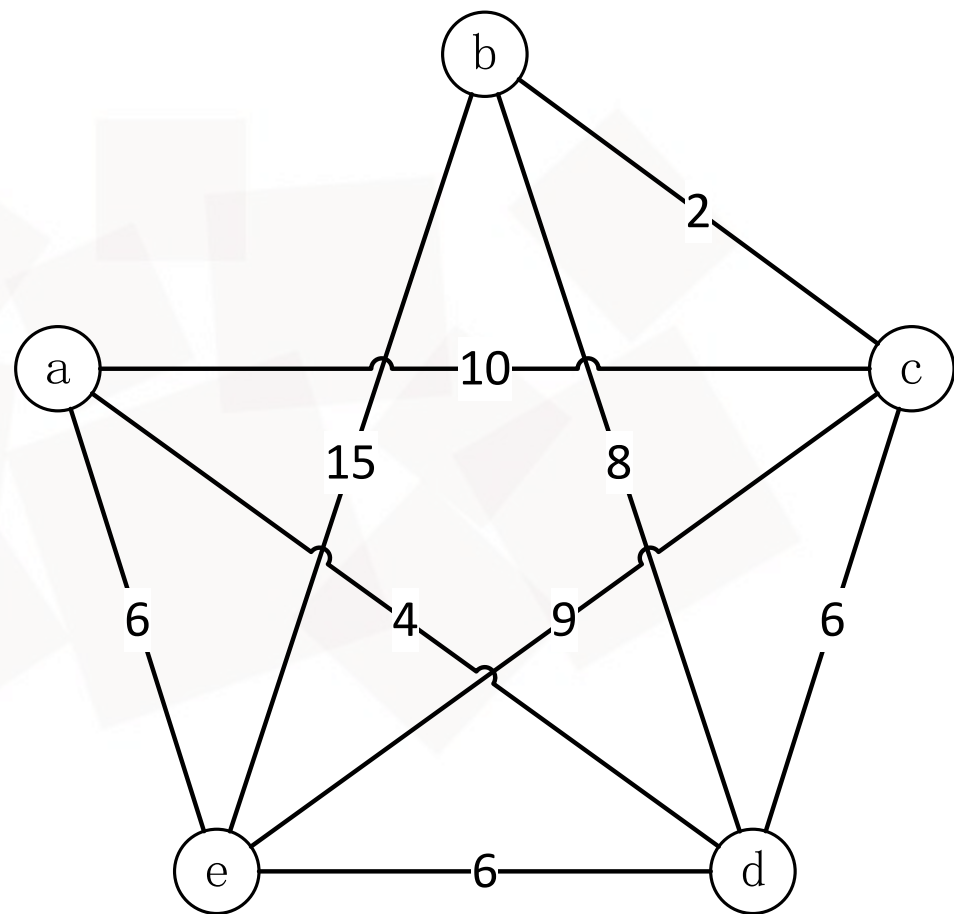
## 随机爬山法

假设从城市a出发，用城市的序列表示该问题的一个可能解

- 初始状态: 随机选择初始可能解

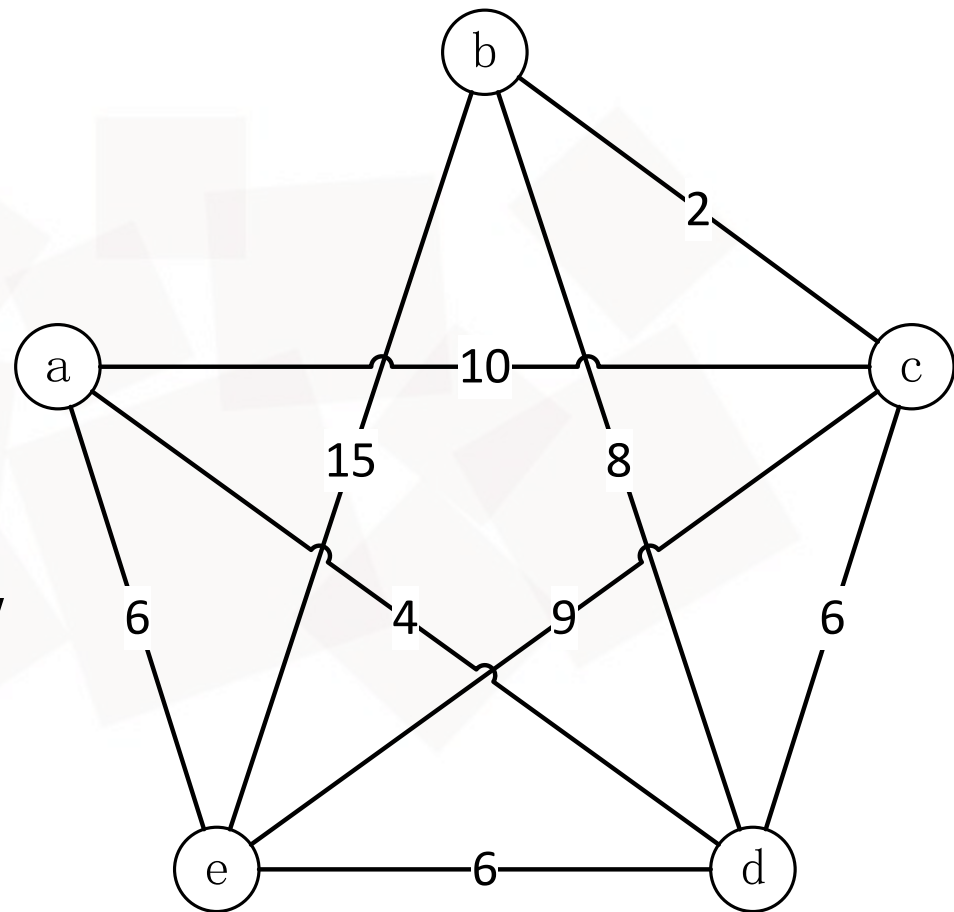
$$x_0 = abcdea$$

$$f(x_0) = +\infty$$



## 第1次搜索 (随机爬山法)

- $x_0 = abcdea$  的邻域  
 $P = \{acbdea, adcbea, aecdba, abdcea, abedca, abceda\}$
- 从P中随机选择一个元素  $x_n = acbdea$ ,  
 $f(x_n) = 32 < f(x_0) = +\infty$
- 更新  $x_0 = acbdea$



## 第2次搜索 (随机爬山法)

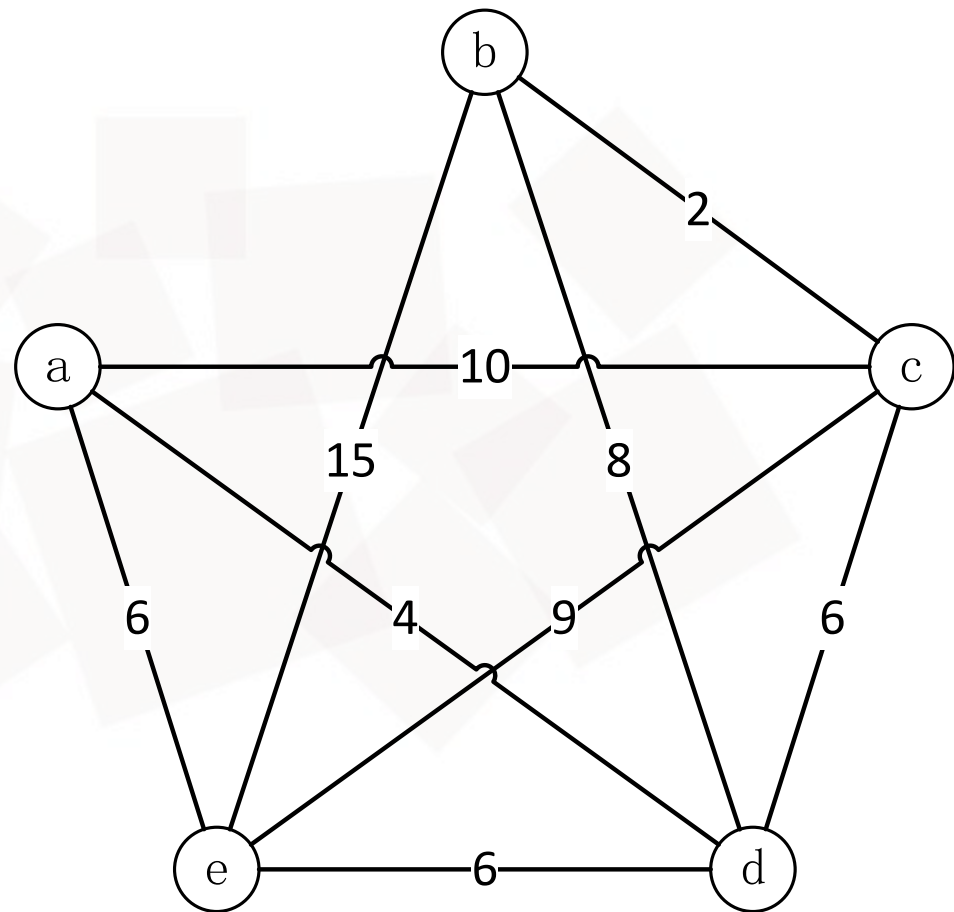
- $x_0 = \text{acbdea}$  的邻域

$P = \{\text{abcdea}, \text{adbcea}, \text{aebdca}, \text{acdbea}, \text{acedba}, \text{acbeda}\}$

从  $P$  中 随机选择 一个元素

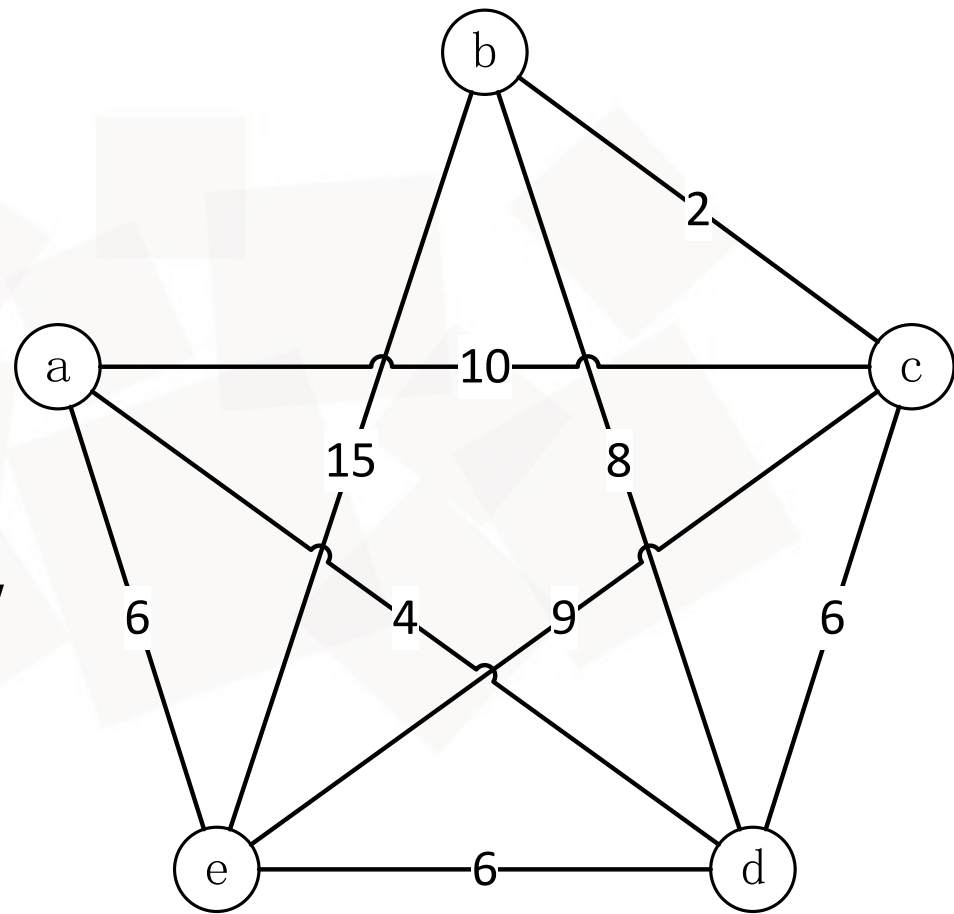
$x_n = \text{abcdea}$ ,  $f(x_n) = +\infty > f(x_0) = 32$

- 更新  $P = \{\text{adbcea}, \text{aebdca}, \text{acdbea}, \text{acedba}, \text{acbeda}\}$



## 第3次搜索 (随机爬山法)

- $x_0 = \text{acbdea}$  的邻域  
 $P = \{\text{adbcea}, \text{aebdca}, \text{acdbea}, \text{acedba}, \text{acbeda}\}$
- 从P中随机选择一个元素  $x_n = \text{adbcea}$ ,  
 $f(x_n) = 29 < f(x_0) = 32$
- 更新  $x_0 = \text{adbcea}$

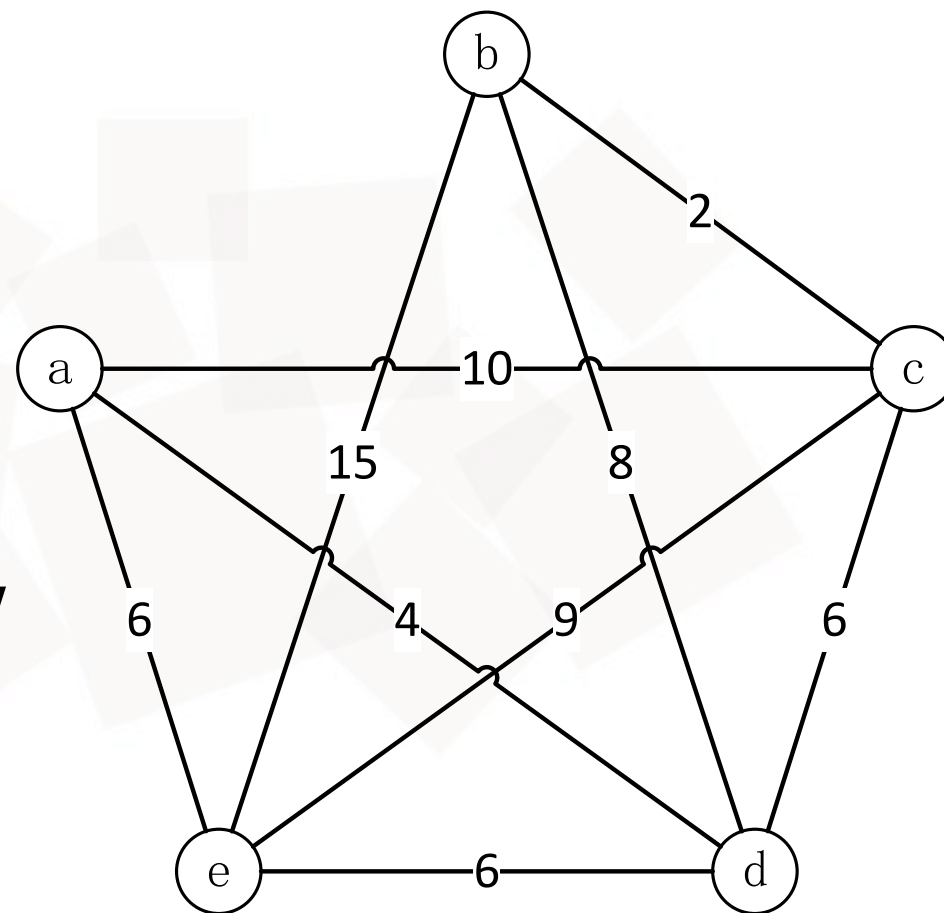






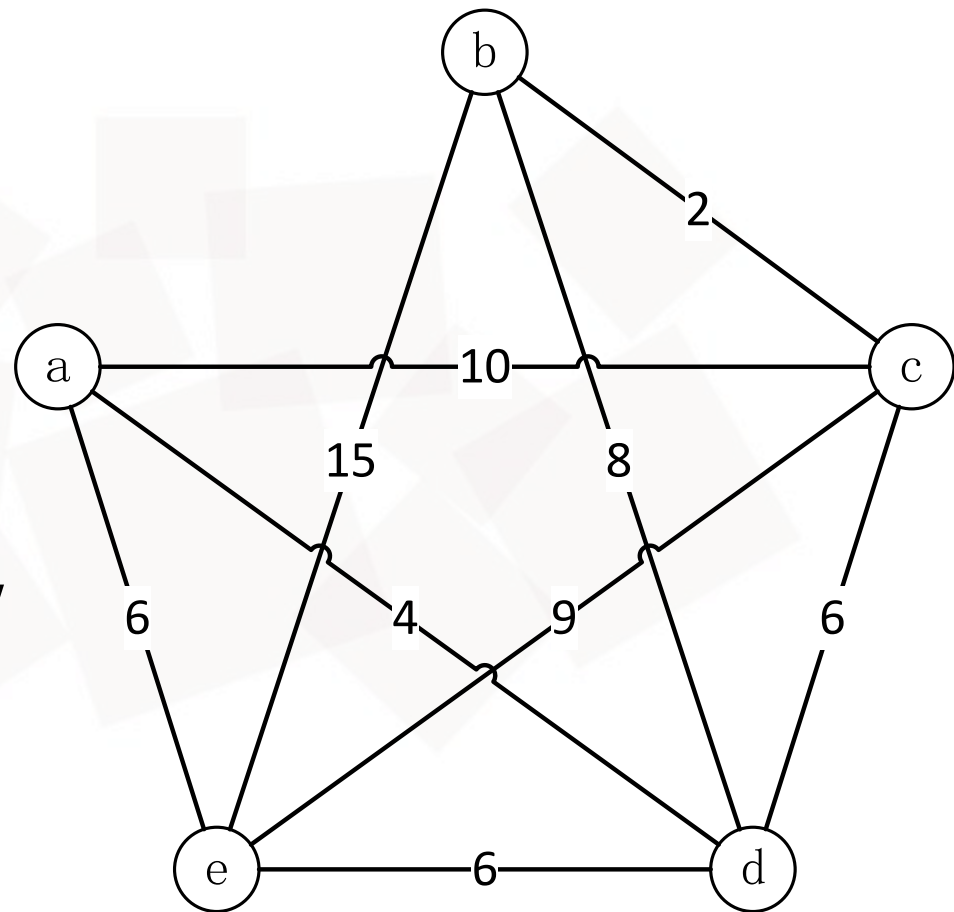
## 第4次搜索（随机爬山法）

- $x_0 = \text{adbcea}$  的邻域  
 $P = \{\text{abdcea}, \text{acbdea}, \text{aebcda}, \text{adcbea}, \text{adecba}, \text{adbeca}\}$
- 从P中随机选择一个元素  $x_n = \text{abdcea}$ ,  
 $f(x_n) = +\infty > f(x_0) = 29$
- 更新  $P = \{\text{acbdea}, \text{aebcda}, \text{adcbea}, \text{adecba}, \text{adbeca}\}$



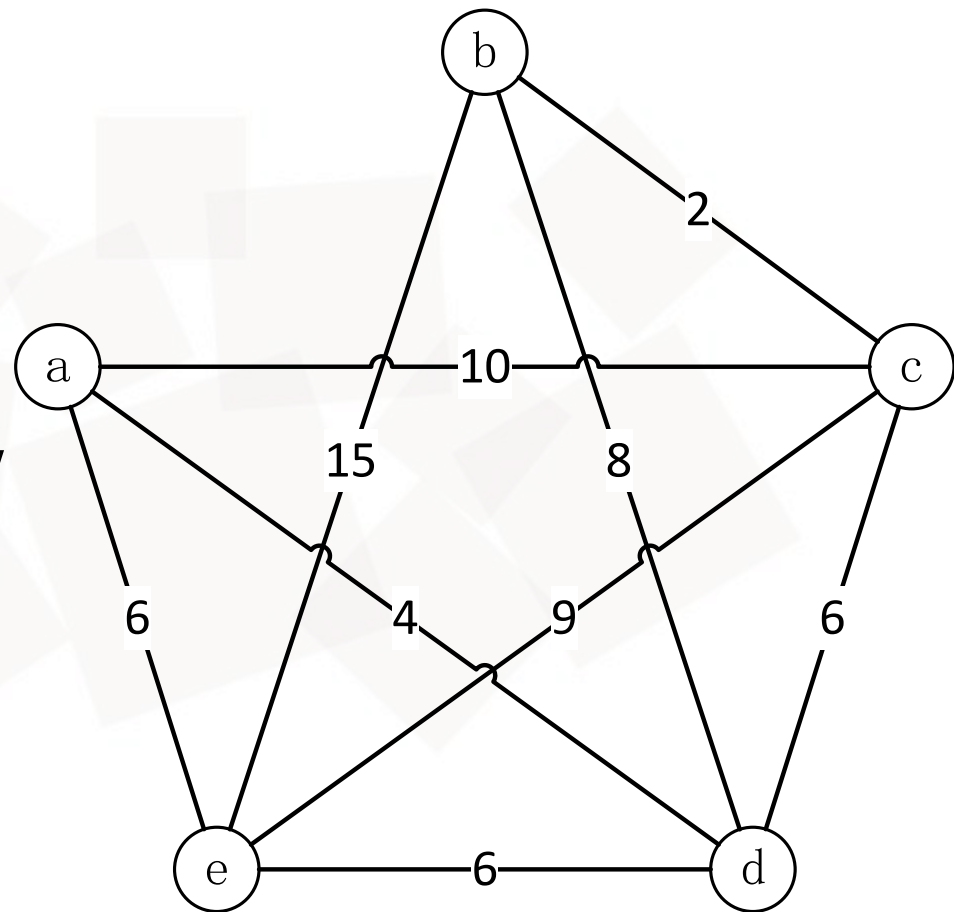
## 第5次搜索 (随机爬山法)

- $x_0 = \text{adbcea}$  的邻域  
 $P = \{\text{acbdea}, \text{aebcda}, \text{adcbea}, \text{adecba}, \text{adbeca}\}$
- 从P中随机选择一个元素  $x_n = \text{acbdea}$ ,  
 $f(x_n) = 32 > f(x_0) = 29$
- 更新  $P = \{\text{aebcda}, \text{adcbea}, \text{adecba}, \text{adbeca}\}$



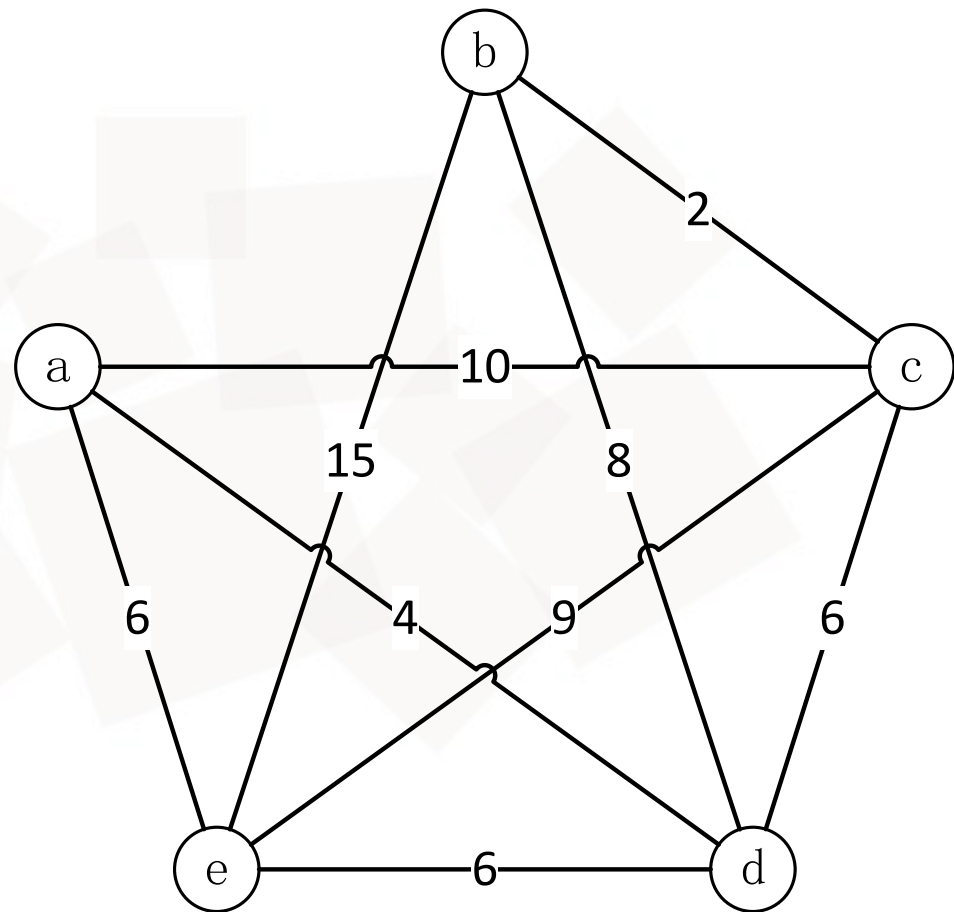
## 第6次搜索 (随机爬山法)

- $x_0 = \text{adbcea}$  的邻域  
 $P = \{\text{aebcda}, \text{adcbea}, \text{adecba}, \text{adbeca}\}$
- 从P中随机选择一个元素  $x_n = \text{aebcda}$ ,  
 $f(x_n) = 33 > f(x_0) = 29$
- 更新  $P = \{\text{adcbea}, \text{adecba}, \text{adbeca}\}$



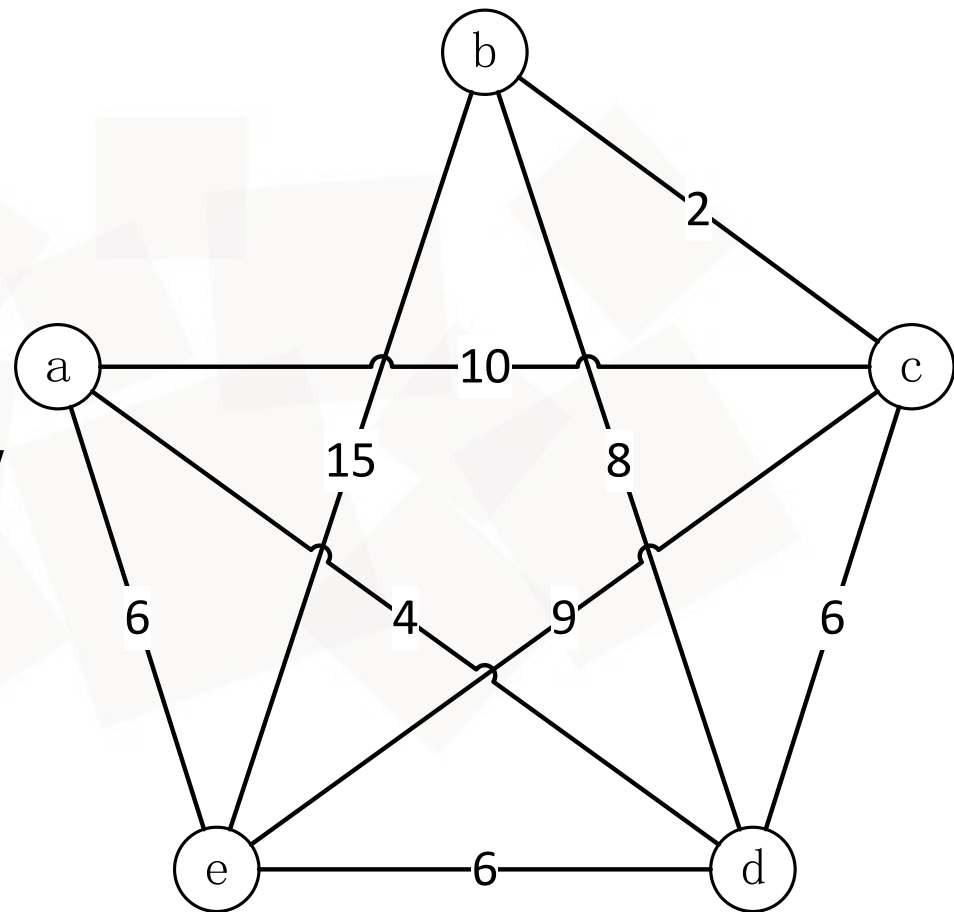
## 第7次搜索 (随机爬山法)

- $x_0 = \text{adbcea}$  的邻域  
 $P = \{\text{adcbea}, \text{adecba}, \text{adbeca}\}$
- 从  $P$  中选择 随机选择 一个元素  
 $x_n = \text{adcbea}$ ,  $f(x_n) = 33 > f(x_0) = 29$
- 更新  $P = \{\text{adecba}, \text{adbeca}\}$



## 第8次搜索 (随机爬山法)

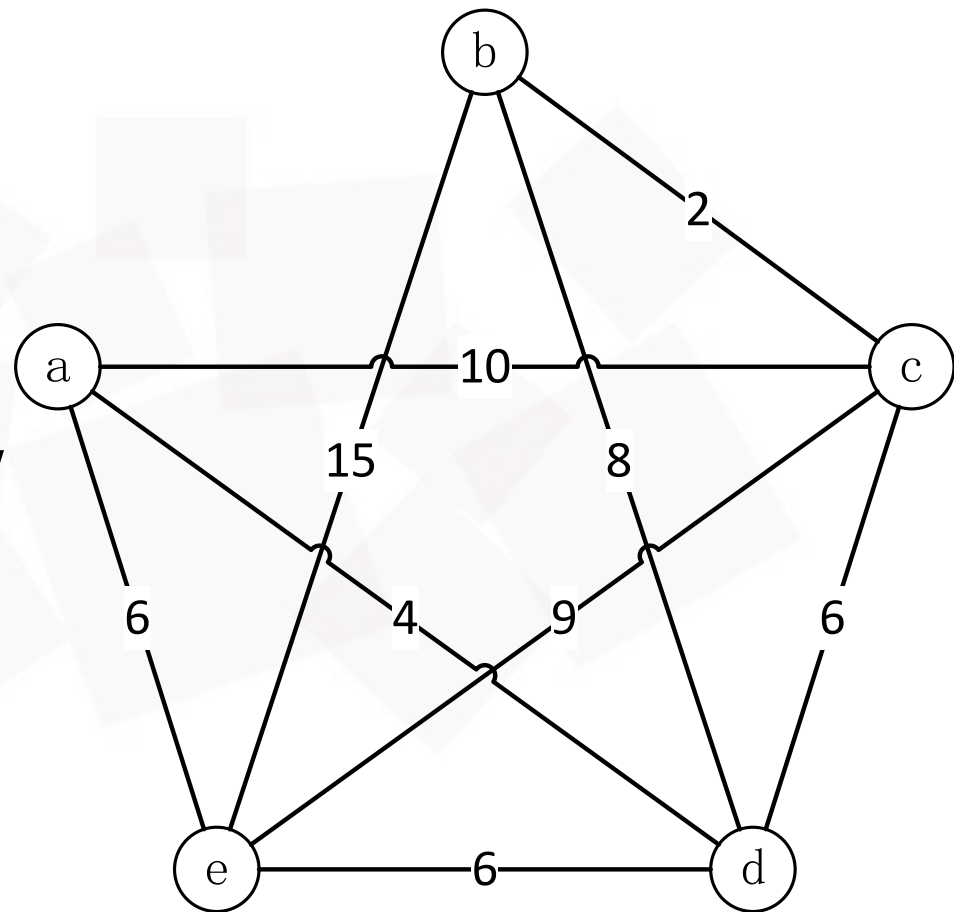
- $x_0 = \text{adbcea}$  的邻域  
 $P = \{\text{adecba}, \text{adbeca}\}$
- 从P中随机选择一个元素  $x_n = \text{adecba}$ ,  
 $f(x_n) = +\infty > f(x_0) = 29$
- 更新  $P = \{\text{adbeca}\}$



## 第9次搜索 (随机爬山法)

- $x_0 = \text{adbcea}$  的邻域  
 $P = \{\text{adbeca}\}$
- 从P中随机选择一个元素  $x_n = \text{adbeca}$ ,  
 $f(x_n) = 46 > f(x_0) = 29$
- 更新  $P = \{\}$ , 算法结束
- 搜索结果

$$x_0 = \text{adbcea}, \quad f(x_0) = 29$$





## 模拟退火搜索

- 退火：冶金学中，通过将金属和玻璃加热到高温然后逐渐冷却的方法，使材料达到低能量结晶态以进行回火或硬化的过程
- 类似将乒乓球放入一个崎岖表面的最深裂缝中
  - 如果只是让球滚动，它会停在一个局部极小值
  - 如果晃动平面，乒乓球会从局部极小值中弹出来，也许会落入更深的局部极小值
  - 需要控制晃动幅度：晃动足够小以使球从局部极小值中弹出，但又不能太大使其无法从全局最小值弹出





## 模拟退火搜索

- 高温加热：猛烈晃动
- 逐渐降温：逐渐降低晃动幅度
- 允许“暂时接受劣解”：与传统爬山法不同，模拟退火在迭代过程中可以以一定概率接受比当前解更差的解，从而避免陷入局部最优
- 温度参数（ $T$ ）控制随机性：
  - 高温阶段：算法倾向于接受更多劣解，广泛探索解空间
  - 低温阶段：算法逐渐收敛，类似传统爬山法，仅在局部改进



## 模拟退火搜索

- 后继状态的目标函数值更优，则接受，否则以小于1的概率接受

**function** SIMULATED-ANNEALING(*problem*, *schedule*) **returns** 一个解状态

*current*  $\leftarrow$  *problem*.INITIAL

**for**  $t = 1$  **to**  $\infty$  **do**

*T*  $\leftarrow$  *schedule*( $t$ )

**if**  $T = 0$  **then return** *current*

*next*  $\leftarrow$  *current*的一个随机选择的后继状态

$\Delta E \leftarrow \text{VALUE}(\text{current}) - \text{VALUE}(\text{next})$

**if**  $\Delta E > 0$  **then** *current*  $\leftarrow$  *next*

**else** *current*  $\leftarrow$  *next*仅以 $e^{\Delta E/T}$ 的概率



## 局部束搜索

- 记录 $k$ 个状态
- 从 $k$ 个随机生成的状态开始，每步生成 $k$ 个状态的所有后继状态。如果其中任意一个是目标状态，算法停止；否则只选取 $k$ 个最佳后继状态，重复上述过程
- 与 $k$ 个随机重启不同：
  - 随机重启，每个搜索进程独立运行
  - 局部束搜索，有用信息将在并行的搜索线程之间传递，算法将很快放弃那些没有效果的搜索并把资源转移到取得最大进展的路径上



## 随机束搜索

- 局部束搜索的变体
- 随机选取 $k$ 个后继状态，状态的选择概率与状态的目标函数值成正比，增加多样性

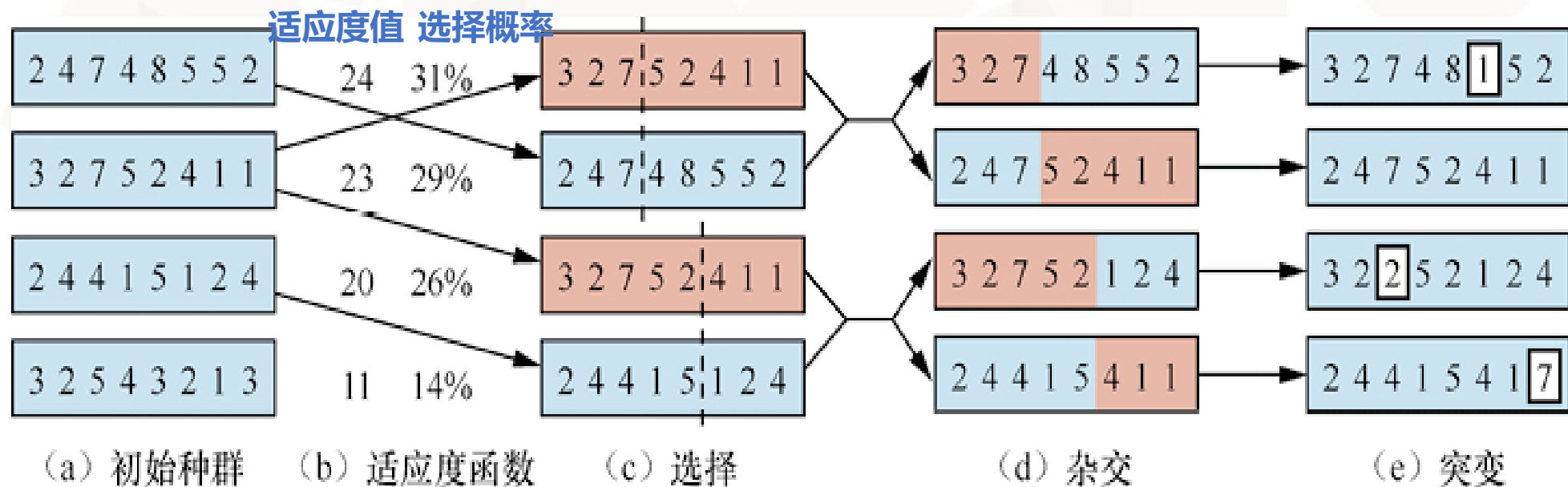


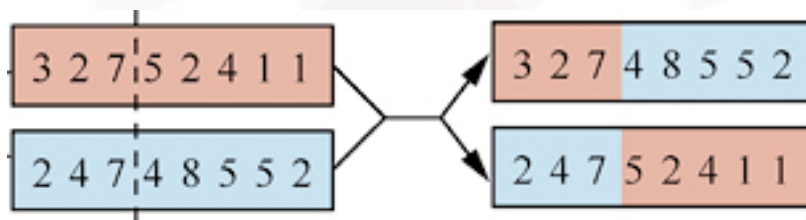
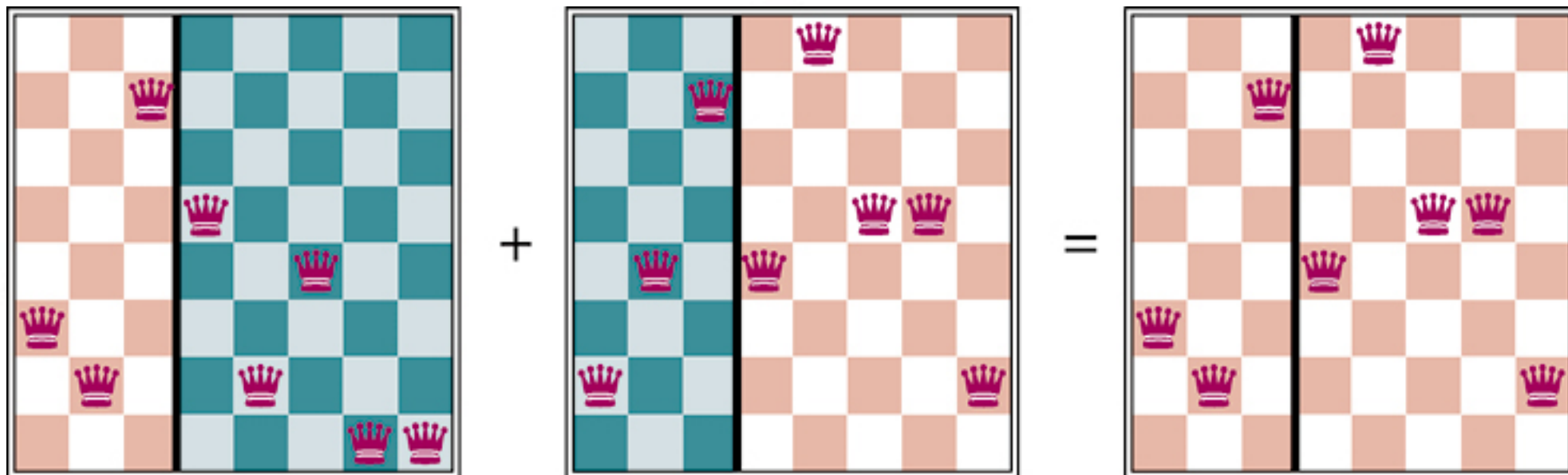
## 随机束搜索的变体——遗传算法

- 生物学中，一个由个体（状态）组成的种群，其中最适应环境（值最高）的个体可以生成后代（后继状态）来繁衍下一代，这个过程被称为重组
- 通过两个状态的结合生成后继状态
- 种群：目前的所有状态，从k个随机状态开始
- 个体：每个状态，有限字母表上的一个字符串
  - 八皇后问题：八位数字字符串
- 适应度函数：目标函数，求全局极大值
  - 八皇后问题：互不攻击的皇后对数，最优解适应度为28

## 遗传算法

- 选择---杂交---突变





(1是最下面一行，8是最上面一行)





**function** GENETIC-ALGORITHM(*population*, *fitness*) **returns** 一个个体

**repeat**

*weights*  $\leftarrow$  WEIGHTED-BY(*population*, *fitness*)

*population2*  $\leftarrow$  空列表

**for** *i* = 1 **to** SIZE(*population*) **do**

*parent1*, *parent2*  $\leftarrow$  WEIGHTED-RANDOM-CHOICES(*population*, *weights*, 2)

*child*  $\leftarrow$  REPRODUCE(*parent1*, *parent2*)

**if** (小的随机概率) **then** *child*  $\leftarrow$  MUTATE(*child*)

将*child*添加到*population2*中

*population*  $\leftarrow$  *population2*

**until** 某个个体足够适应，或者已经经过了足够长的时间

**return** 依据 *fitness* 选出的 *population* 中的最优个体

**function** REPRODUCE(*parent1*, *parent2*) **returns** 一个个体

*n*  $\leftarrow$  LENGTH(*parent1*)

*c*  $\leftarrow$  从1到*n*的随机数

**return** APPEND(SUBSTRING(*parent1*, 1, *c*), SUBSTRING(*parent2*, *c* + 1, *n*))

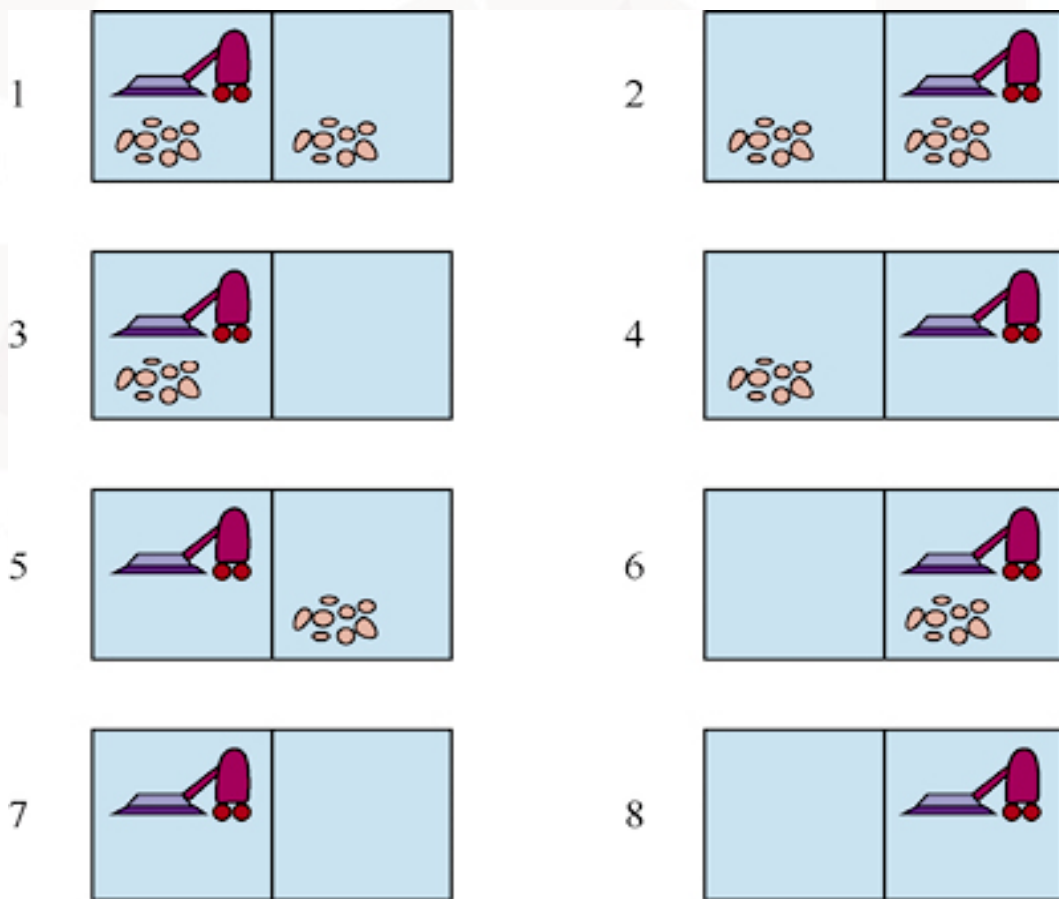


- 精英主义：下一代的构成，可能只包括新形成的后代，也可能包括上一代中得分最高的个体
- 淘汰：丢弃所有分数低于给定阈值的个体，使得进化加速
- 模式：某些位未确定的子串
  - 模式246\*\*\*\*\*表示前三个皇后分别位于位置2、4、6的所有八皇后状态
- 实例：与模式相匹配的字符串
  - 如24613578
- 如果某一模式实例的平均适应度高于平均值，那么该模式的实例数量将随着时间推移而不断增加



- 之前假定环境是完全可观测的、确定性的、已知的
    - “我现在位于 $s_1$ 状态，如果我执行动作 $a$ ，我将会进入 $s_2$ 状态”
  - 使用非确定动作的搜索：当动作是不确定的，感知信息可以确定智能体执行动作的效果
    - “我现在位于 $s_1$ 或 $s_3$ 状态，如果我执行动作 $a$ ，我将会进入 $s_2$ 、 $s_4$ 或 $s_5$ 状态”
- 信念状态**：智能体认为其可能位于的物理状态集合

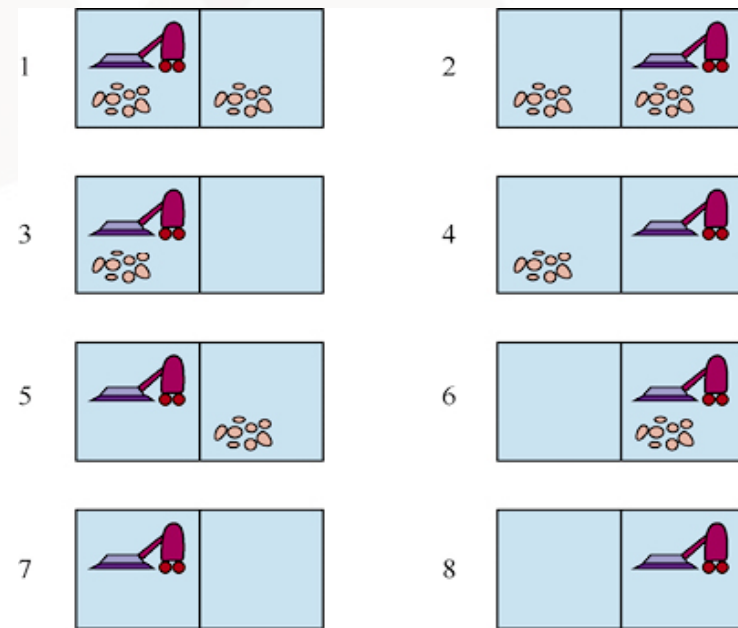
例. 不稳定的真空吸尘器世界



例. 不稳定的真空吸尘器世界的不确定动作

- 在一个不干净的方格执行Suck动作，会清理这一方格，有时也清理它的相邻方格
- 在一个干净的房间执行Suck动作，有时反而会把灰尘弄到地面上

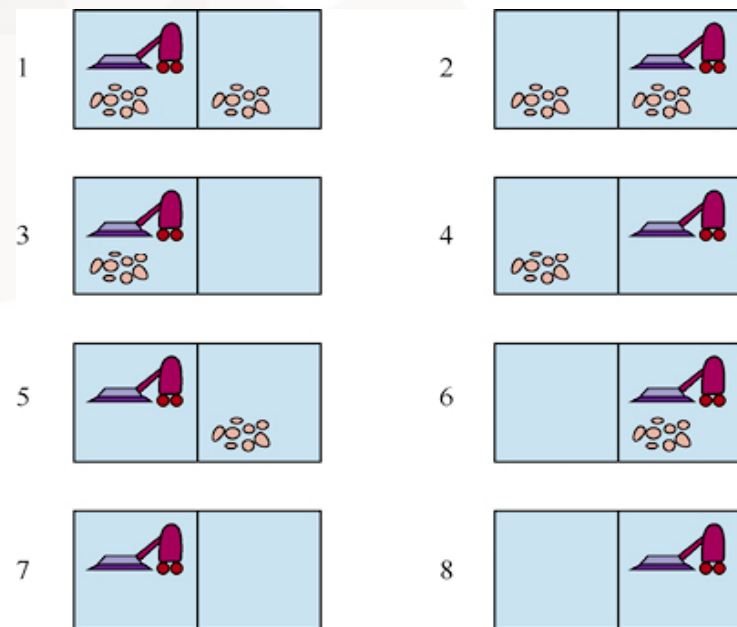
- 转移模型：  $\text{Result}(s, a)$ 
  - 返回多个可能状态，而不再是单个状态
  - $\text{Result}(1, \text{Suck}) = \{5, 7\}$



- 问题的解：条件规划（应变规划或策略），根据智能体在执行规划时接收到的感知来指定动作

- 嵌套的 if-then-else 语句
- 是树，而不再是序列
- [Suck, if State=5 then [Right, Suck] else []]

测试当前状态：智能体运行时  
能够观测到，但规划时不知道



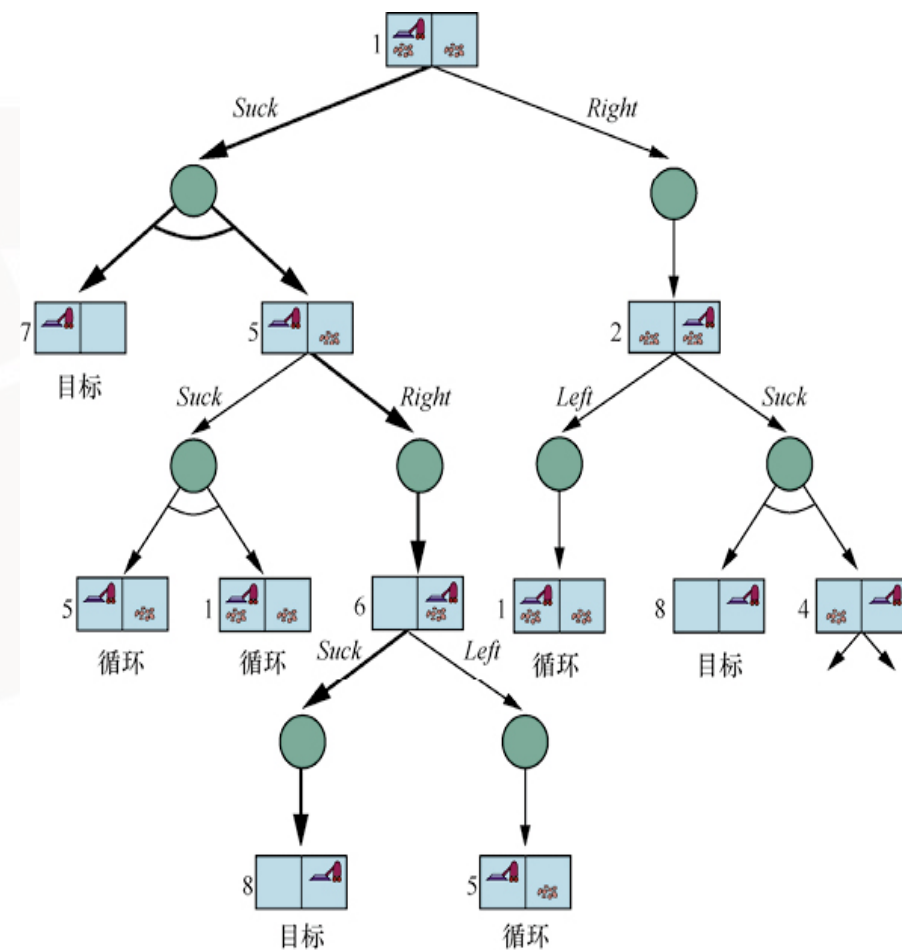


- 与或搜索树

- 或节点：每个状态执行不同的可选动作
  - 与节点：每个动作生成不同的可能状态

- 与或搜索问题的解：一棵子树

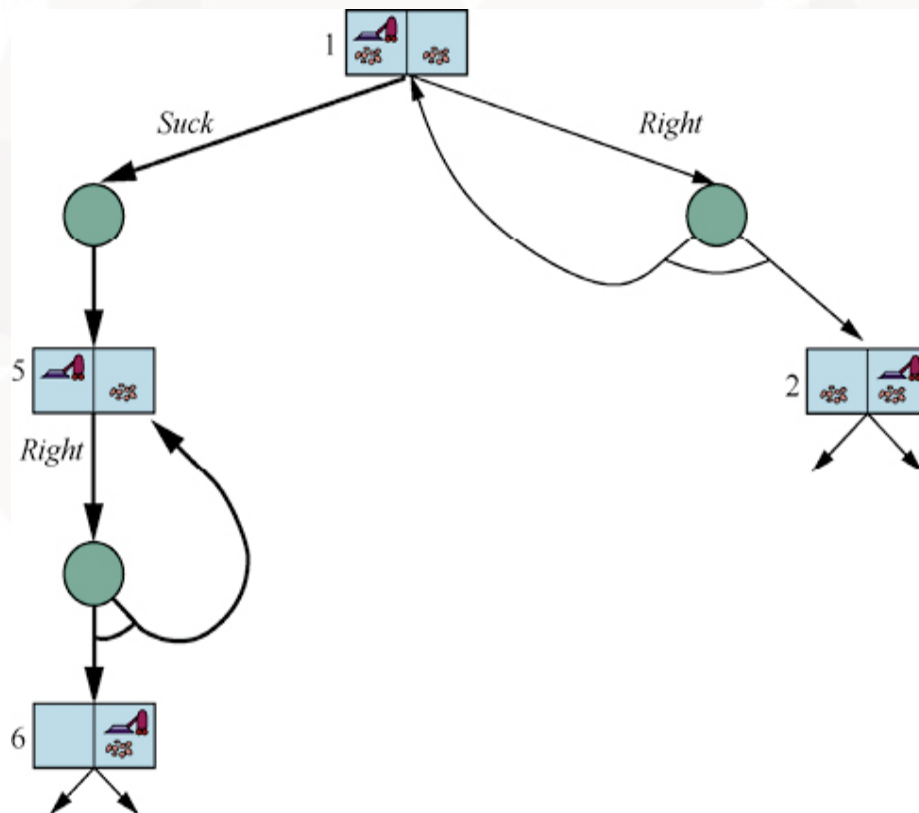
- 每个叶节点为目标节点
  - 或节点选择一个可选动作
  - 与节点选择所有可能状态



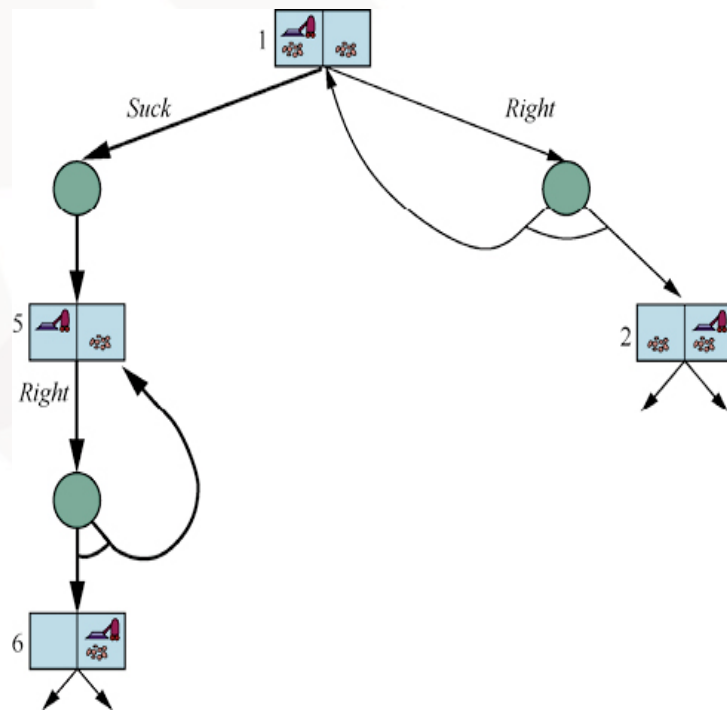


## 例. 光滑的真空吸尘器世界的不确定动作

- 执行Left或Right动作有时会失败，智能体在原地不动



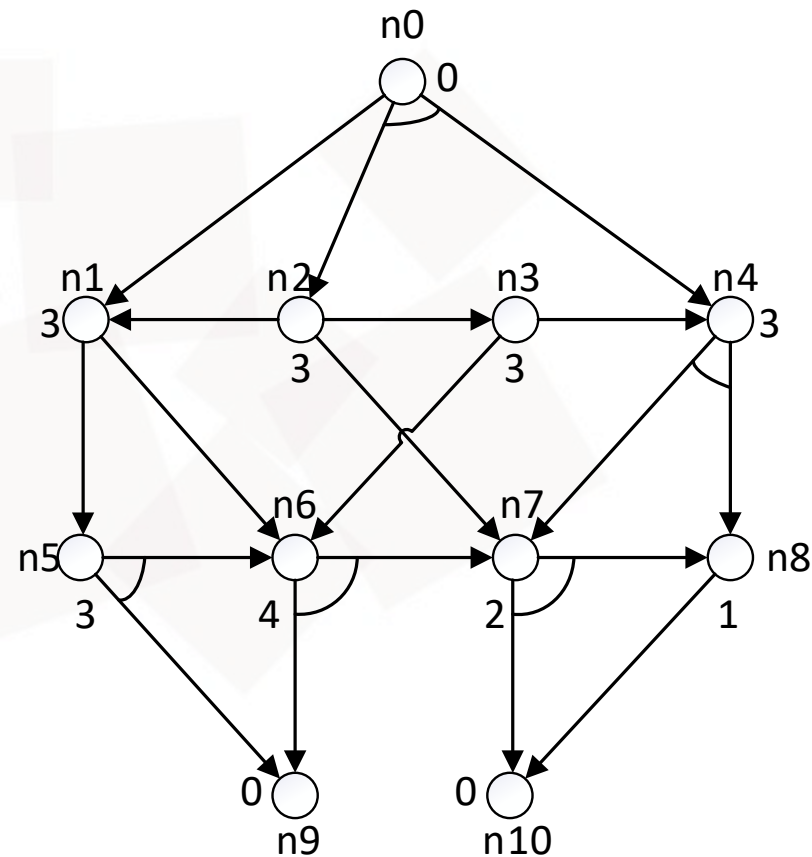
- 循环解：
  - [Suck, L1: Right, **if** State=5 **then** L1 **else** Suck]
  - [Suck, **while** State=5 **do** Right, Suck]
- 循环解作为规划的解
  - 每个叶节点都是一个目标状态
  - 如果动作重复多次，最终总会生效





## 与或图

- 结点间通过K-连接符连接
- $K=1$ 代表确定动作； $K>1$ 代表不确定动作
- 目标：找到从初始结点到达目标结点的最短路径
  - 初始状态：n0
  - 目标状态：n9和n10





## 与或图搜索 深度优先递归算法

**function** AND-OR-SEARCH(*problem*) **returns** 一个条件规划或 *failure*  
**return** OR-SEARCH(*problem*, *problem*.INITIAL, [])

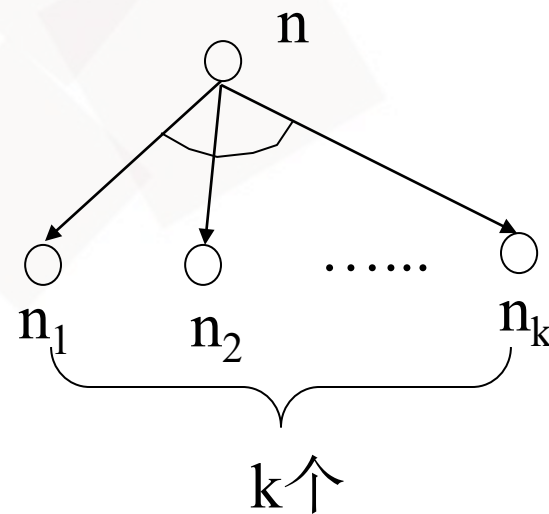
**function** OR-SEARCH(*problem*, *state*, *path*) **returns** 一个条件规划或 *failure*  
**if** *problem*.IS-GOAL(*state*) **then return** 空规划  
**if** IS-CYCLE(*state*, *path*) **then return failure**  
**for each** *action* **在** *problem*.ACTIONS(*state*) **中 do**  
     *plan*  $\leftarrow$  AND-SEARCH(*problem*, RESULTS(*state*, *action*), [*state*] + [*path*])  
     **if** *plan*  $\neq$  *failure* **then return** [*action*] + [*plan*]  
**return failure**

**function** AND-SEARCH(*problem*, *states*, *path*) **returns** 一个条件规划或 *failure*  
**for each**  $s_i$  **in** *states* **do**  
      $plan_i \leftarrow$  OR-SEARCH(*problem*,  $s_i$ , *path*)  
     **if**  $plan_i = failure$  **then return failure**  
**return** [if  $s_1$  **then**  $plan_1$  **else if**  $s_2$  **then**  $plan_2$  **else**  $\cdots$  **if**  $s_{n-1}$  **then**  $plan_{n-1}$  **else**  $plan_n$ ]



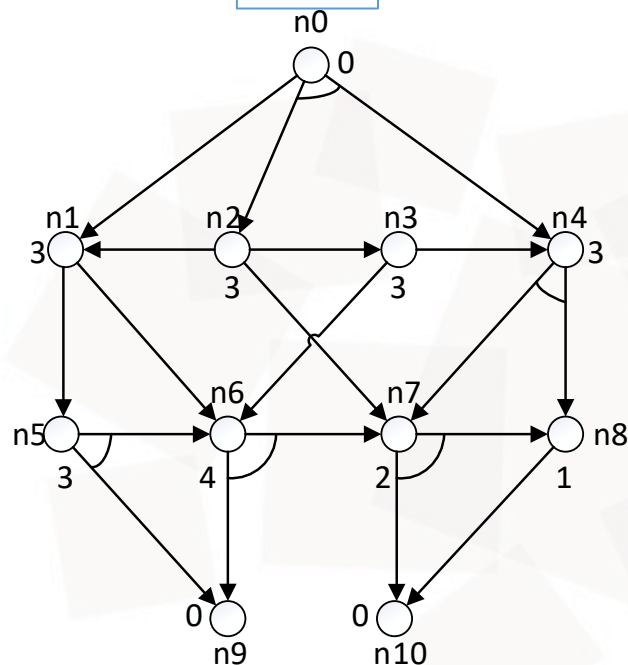
## 与或图的启发式搜索算法 AO\*

- 评估函数 $h(n)$ : 从当前结点 $n$ 到目标结点的最优解的估计代价
- 结点 $n$ 执行 $k$ -连接符对应动作的估计代价为
$$h(n) = C_k + h(n_1) + \dots + h(n_k)$$
  - $C_k$ 为 $k$ -连接符的实际代价
  - 可能存在多个目标结点
- “自上向下”扩展节点, “自下向上”更新代价

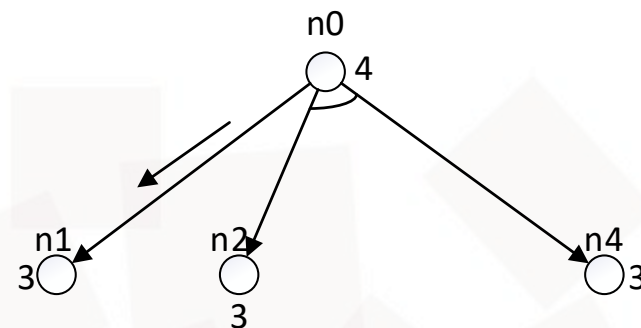


## 第一次扩展

原图

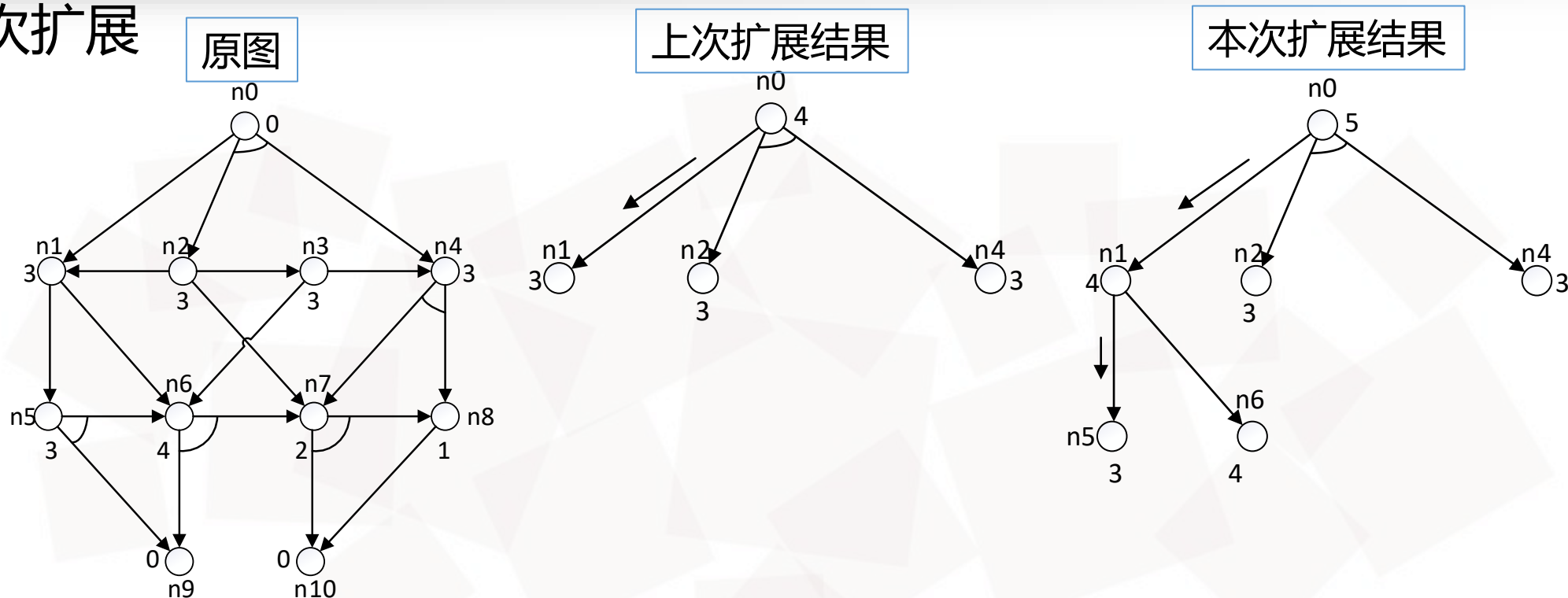


本次扩展结果



- 扩展初始结点 $n_0$ ，左侧的1-连接符为最优动作
- $n_0$ 的代价更新为4，当前解路径的 $n_1$ 的代价最小为待扩展结点

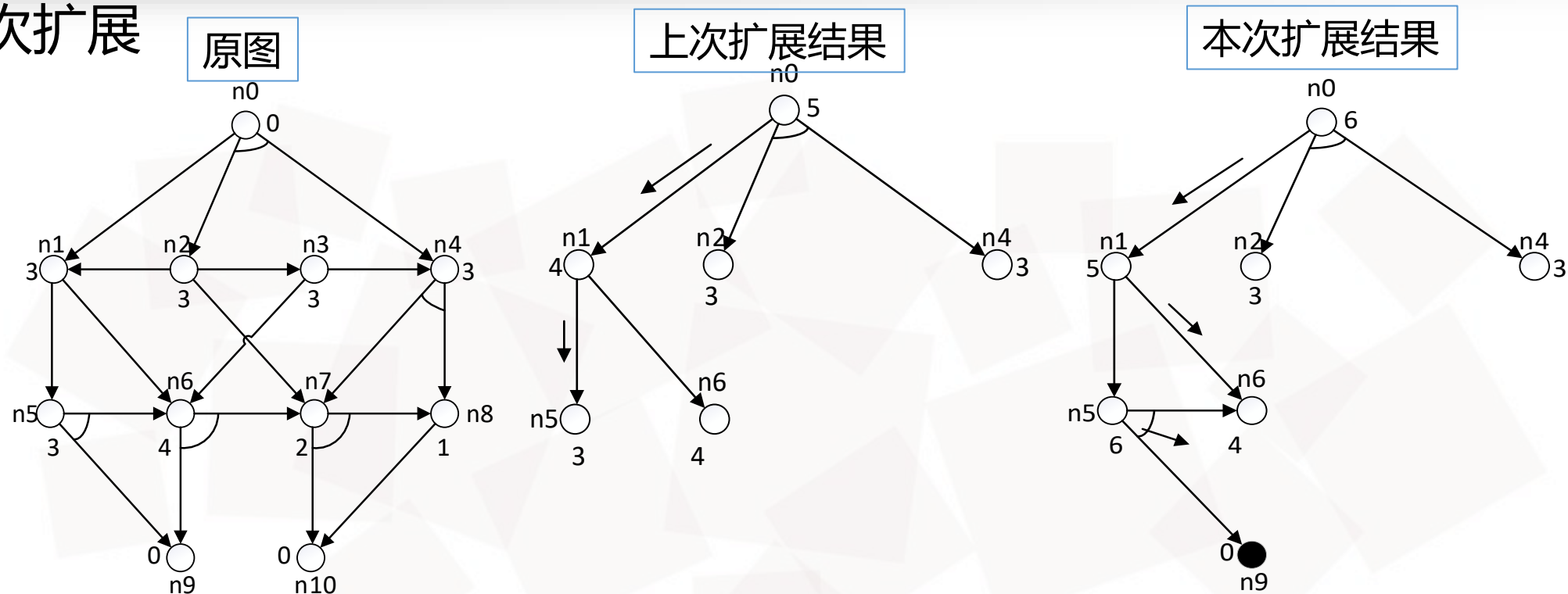
## 第二次扩展



- 扩展结点n1，左侧的1-连接符为最优动作
- n1的代价更新为4，n0的代价更新为5，当前解路径的n5的代价最小为待扩展结点

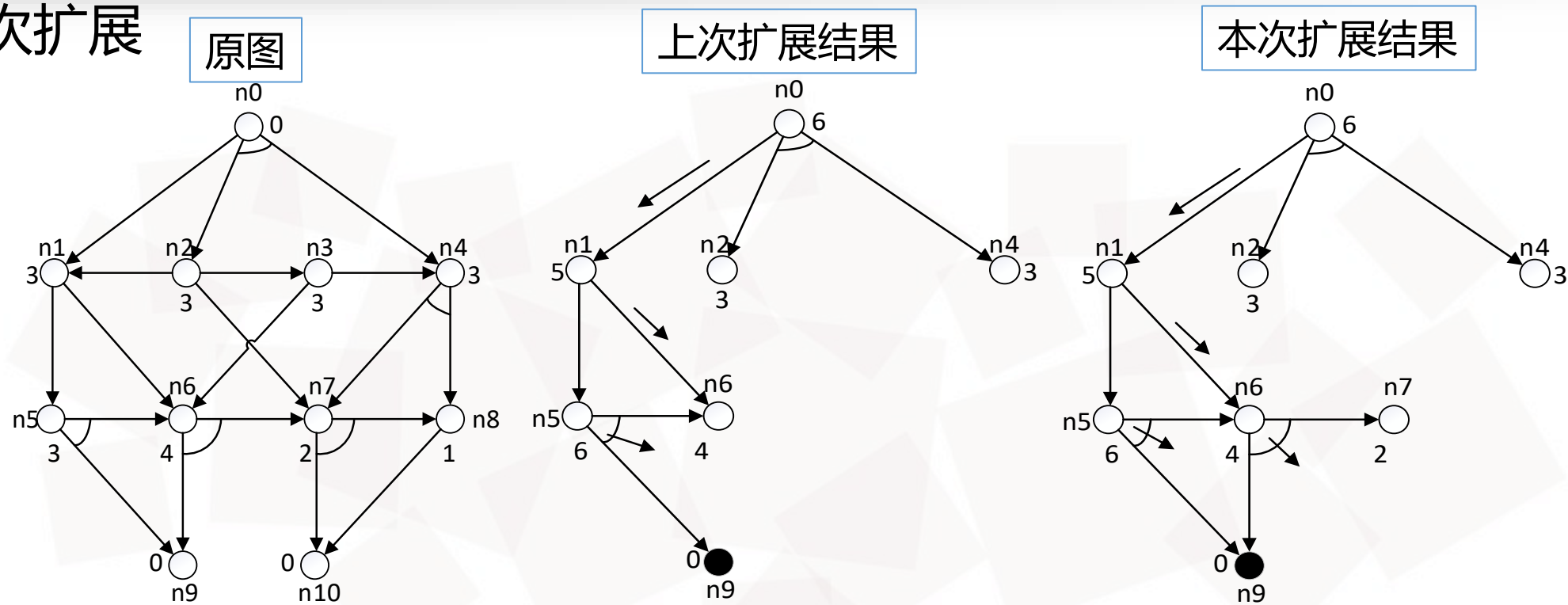


## 第三次扩展



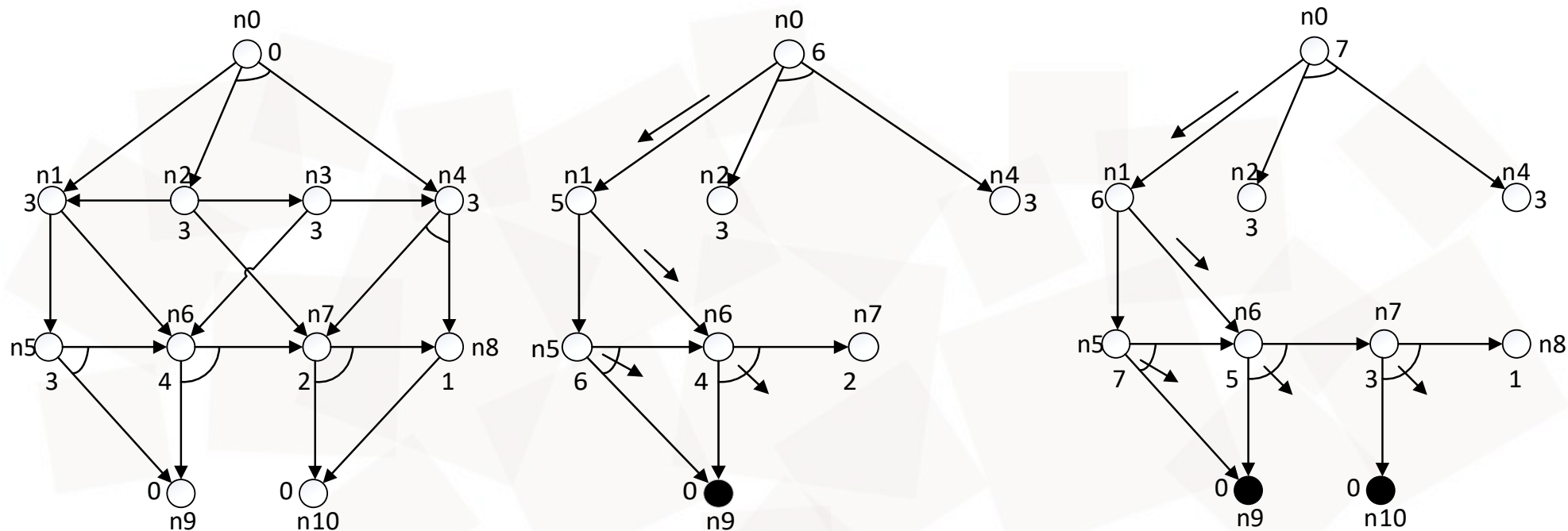
- 扩展结点n5，唯一的2-连接符为最优动作
- n5的代价更新为6，n1更新了最优动作、代价更新为5，n0的代价更新为6，当前解路径的n6的代价最小为待扩展结点

## 第四次扩展



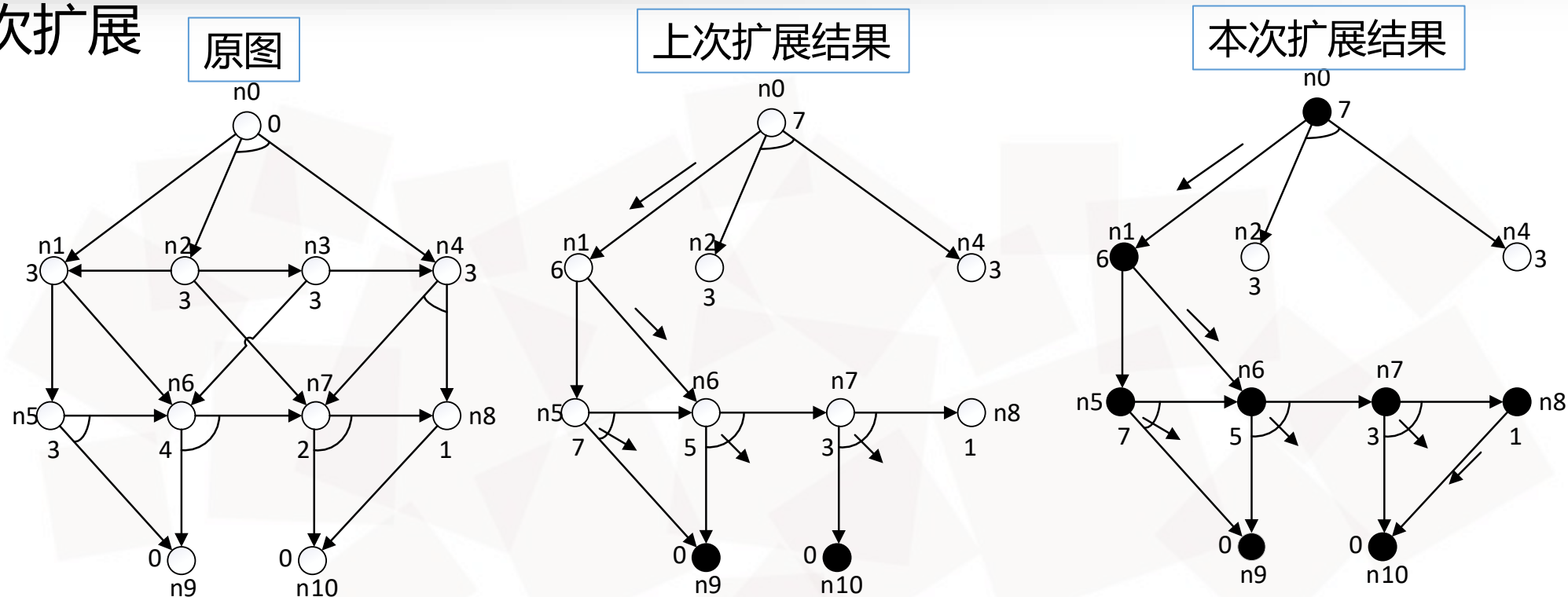
- 扩展结点n6，唯一的2-连接符为最优动作
- n6的代价更新为4（值不变），当前解路径的n7的代价最小为待扩展结点（n9为目标结点）

## 第五次扩展



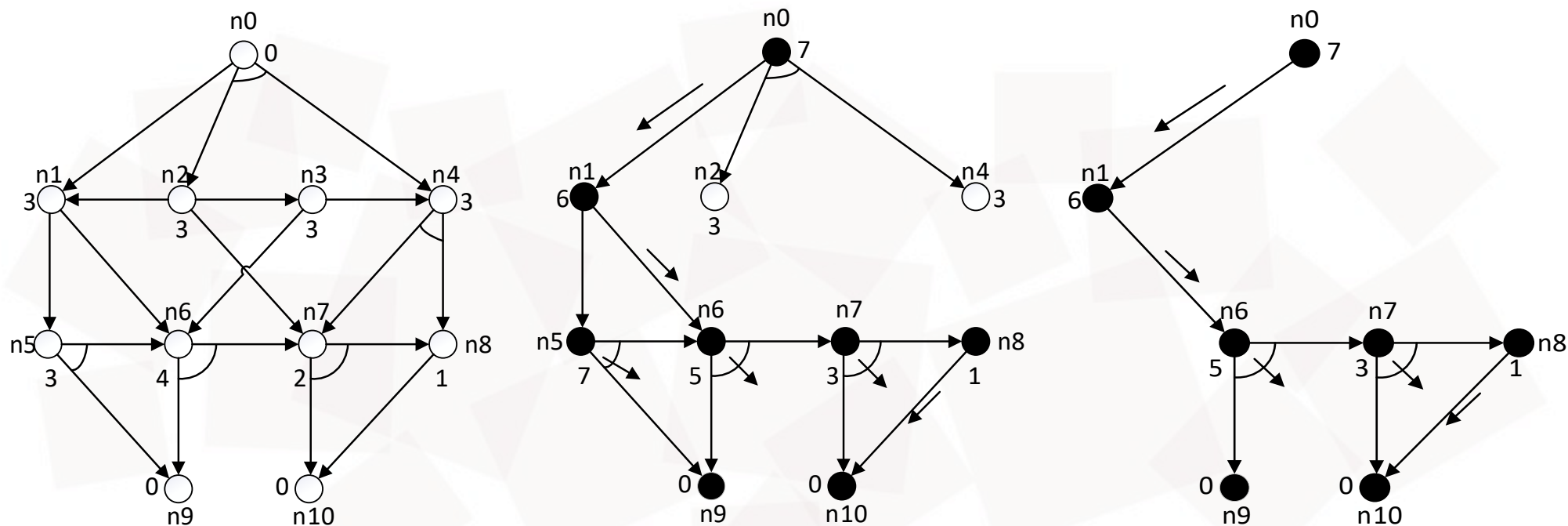
- 扩展结点 $n7$ ，唯一的2-连接符为最优动作
- $n7$ 的代价更新为3， $n6$ 的代价更新为5， $n1$ 、 $n5$ 和 $n0$ 的代价分别更新为6、7、7，当前解路径的 $n8$ 的代价最小为待扩展结点（ $n10$ 为目标结点）

## 第六次扩展



- 扩展结点n8，唯一的1-连接符为最优动作
- n8的代价更新为1（值不变），当前解路径上的所有节点均已扩展

## 问题的解



- 左图：问题原图
- 中图：与或图搜索算法的扩展生成图
- 右图：与或图搜索算法的解图



部分可观测环境中的搜索：当环境部分可观测时，智能体并不确定它处于什么状态，需要感知信息来确定环境的状态信息

## 无观测信息的搜索

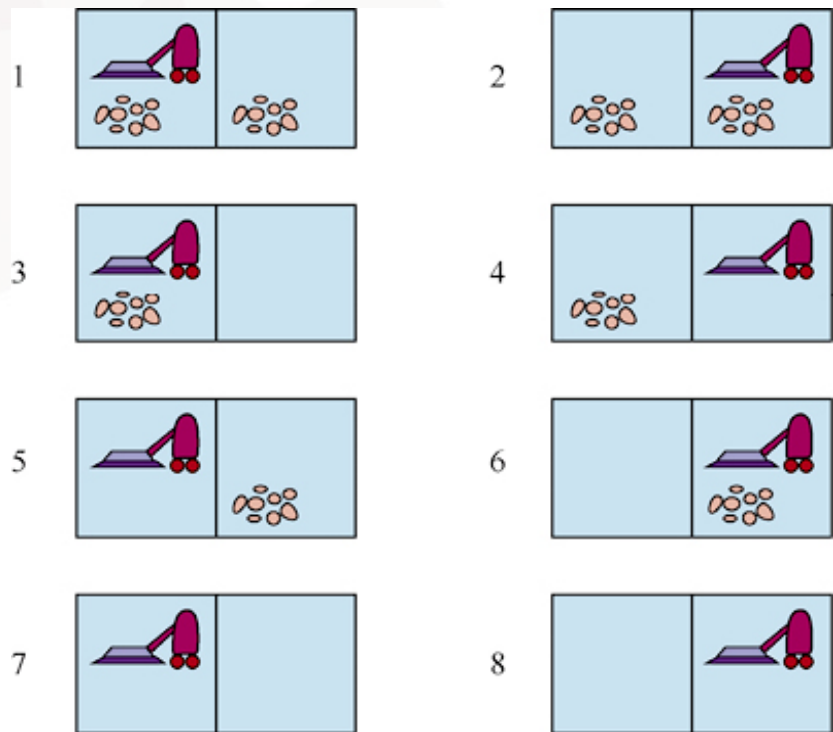
- 无传感器问题：智能体的感知根本不提供任何信息
- 不依赖于传感器是否正常工作
  - 医疗，抗生素避免病情恶化
- 智能体的感知不足以确定准确的状态，智能体的一些动作将致力于减少当前状态的不确定性



## 例. 无观测信息的吸尘器世界

- 智能体知道它所在世界的地理环境，但不知道它自己的位置和灰尘分布
- 初始状态：智能体可能在 $\{1,2,3,4,5,6,7,8\}$ 中任意位置
- 执行Right，智能体可能在 $\{2,4,6,8\}$ 中某个位置
- 执行[Right, Suck]，智能体可能在 $\{4,8\}$ 中某个位置
- 执行[Right,Suck,Left,Suck]，智能体到达目标状态7

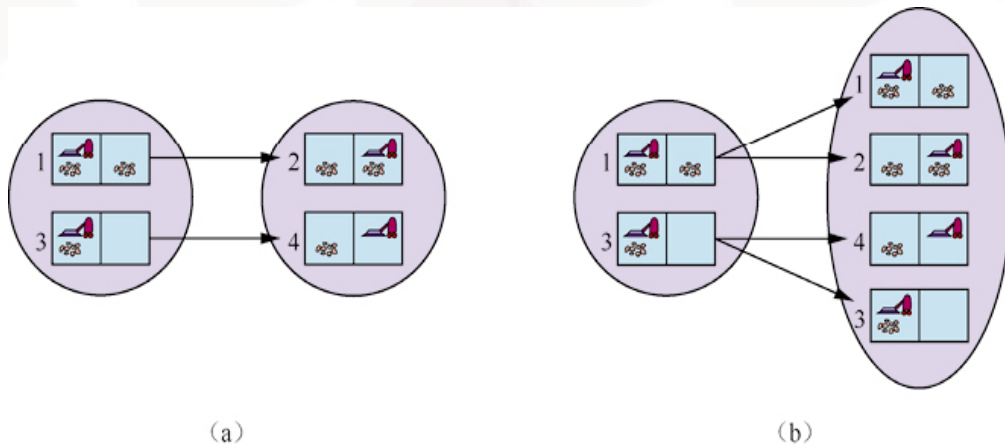
解是动作序列，而非条件规划





## 物理问题P对应的无观测问题

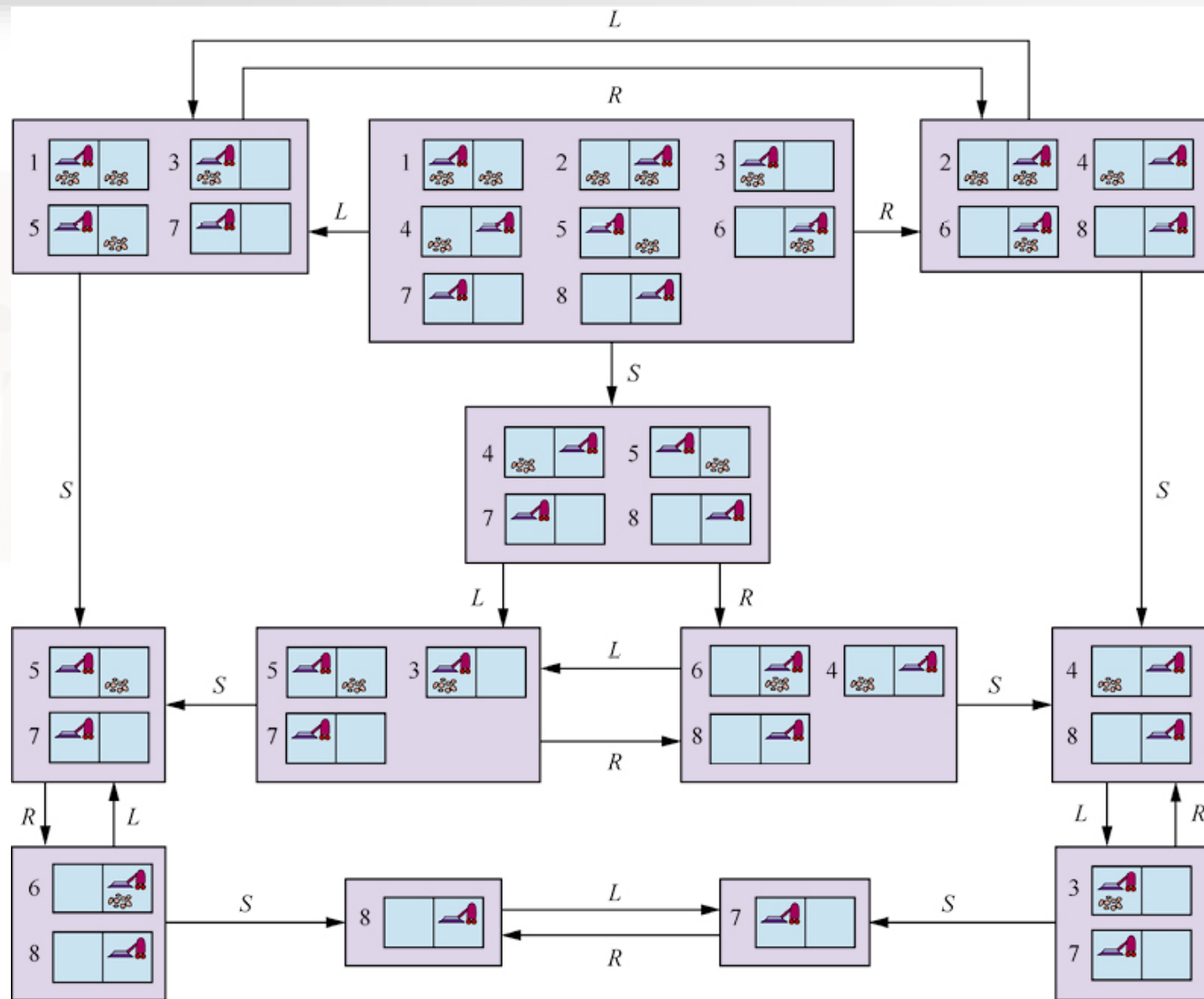
- 信念状态：状态空间包含物理状态的每一个可能子集，P的N个状态对应 $2^N$ 个信念状态
- 初始状态：P的所有状态的集合
- 动作：在不同状态上的同一动作是否合理（并集 / 交集）
- 转移模型：不同状态上分别进行状态转移（确定 / 非确定动作）





## 物理问题P对应的无观测问题

- 目标测试：若信念状态中的任一状态满足底层问题的目标测试，则智能体有**可能**到达了目标；如果所有状态都满足目标测试，则智能体**必定**到达了目标；目标是后者
- 路径代价：同一动作在不同状态下的代价（不同 / 相同）





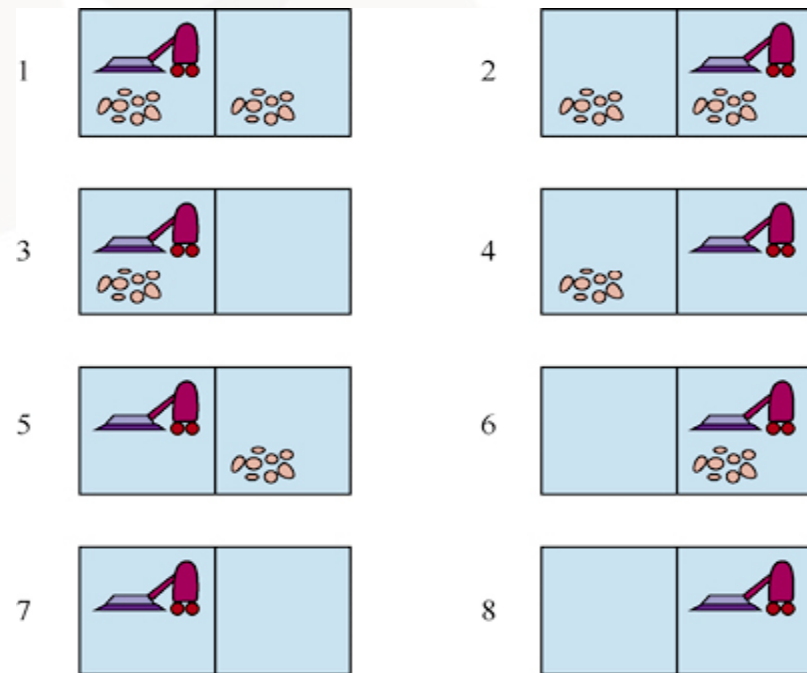
## 增量信念状态搜索

- 每次只为一个物理状态建立解
- 首先求解适合状态1的解
- 然后判断其是否适合状态2，若适合，则继续判断其是否适合状态3，依次下去；若不适合，则回溯求解适合状态1的其它解
- 增量信念状态搜索需要找到一个对所有状态都有效的解
- 增量式能快速检测出失败，当一个信念状态无解，通常情况下它的子集也无解，导致与信念状态规模成正比的加速



## 部分可观测环境中的搜索

- 没有感知就无法求解（八数码问题）
- 一点点感知可能有很大帮助
- $\text{Percept}(s)$ 函数：返回智能体在给定状态下接收的感知
  - 非确定： $\text{Percept}(s)$ 返回可能的感知集合
  - 完全可观测： $\text{Percept}(s)=s$
  - 无传感器： $\text{Percept}(s)=\text{null}$
  - 例：局部感知  $\text{Percept}(\text{State } 1)=[L, \text{Dirty}]$





## 部分可观测问题信念状态之间的转移模型

- 预测：给定信念状态 $b$ 和动作 $a$ ，预测信念状态

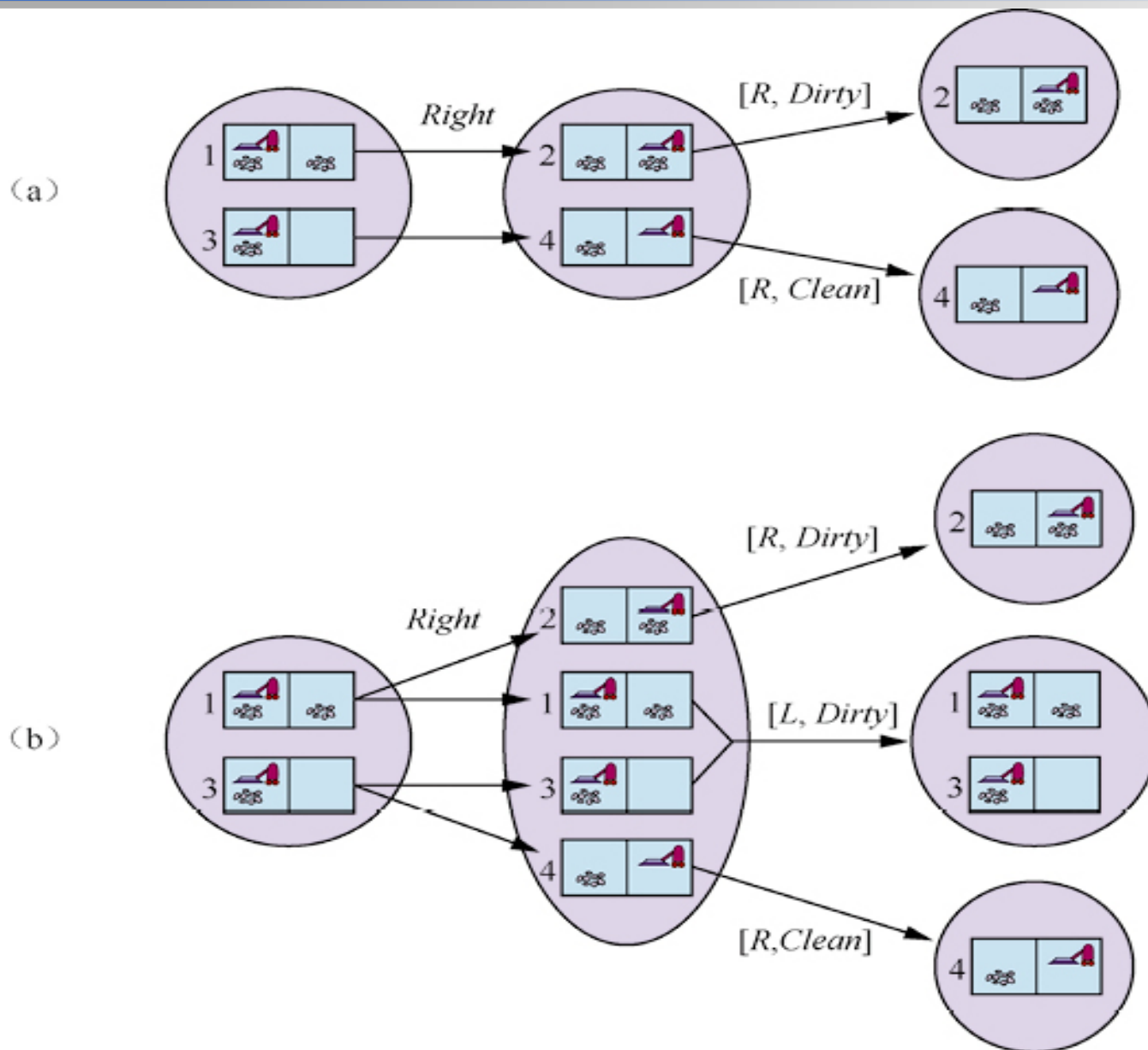
$$\hat{b} = \text{Predict}(b, a)$$

- 可能感知：在预测的信念状态 $\hat{b}$ 下可以观测到的感知集合

$$\text{Possible\_Percept}(\hat{b}) = \{o: o = \text{Percept}(s) \text{ and } s \in \hat{b}\}$$

- 更新：为每个可能感知计算其可能得到的信念状态

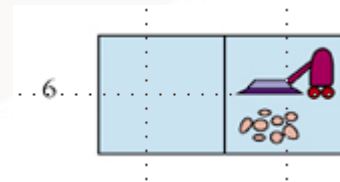
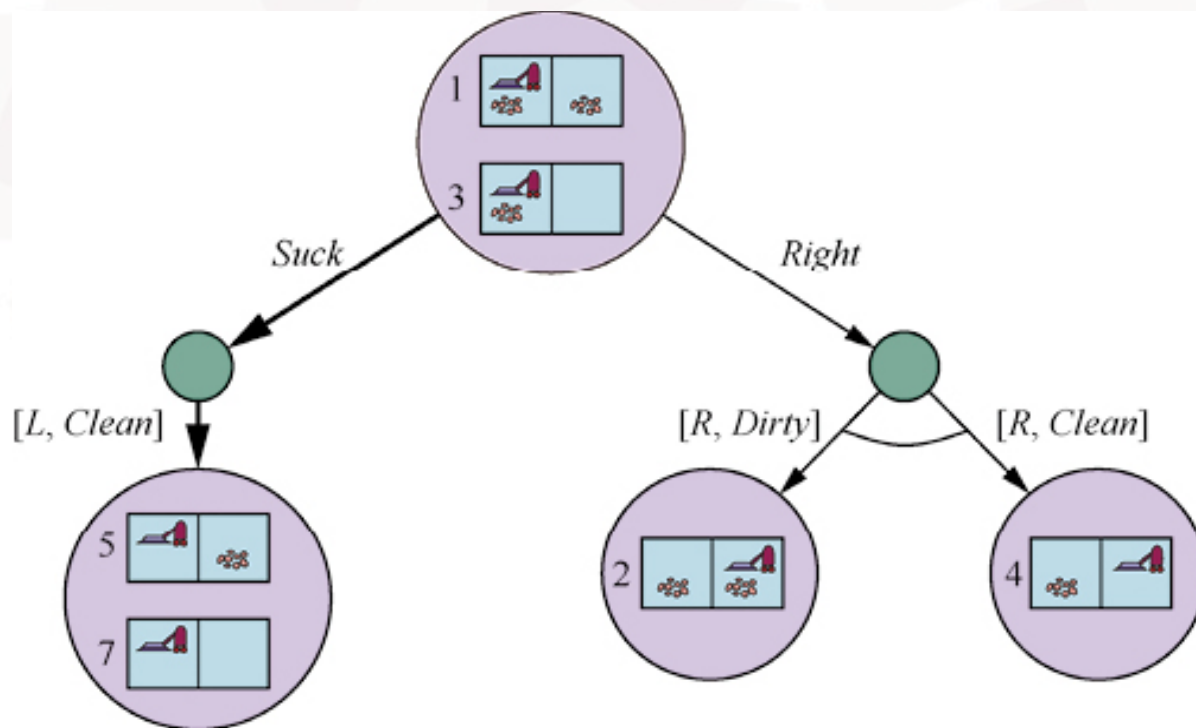
$$b_0 = \text{Update}(\hat{b}, o) = \{s: o = \text{Percept}(s) \text{ and } s \in \hat{b}\}$$





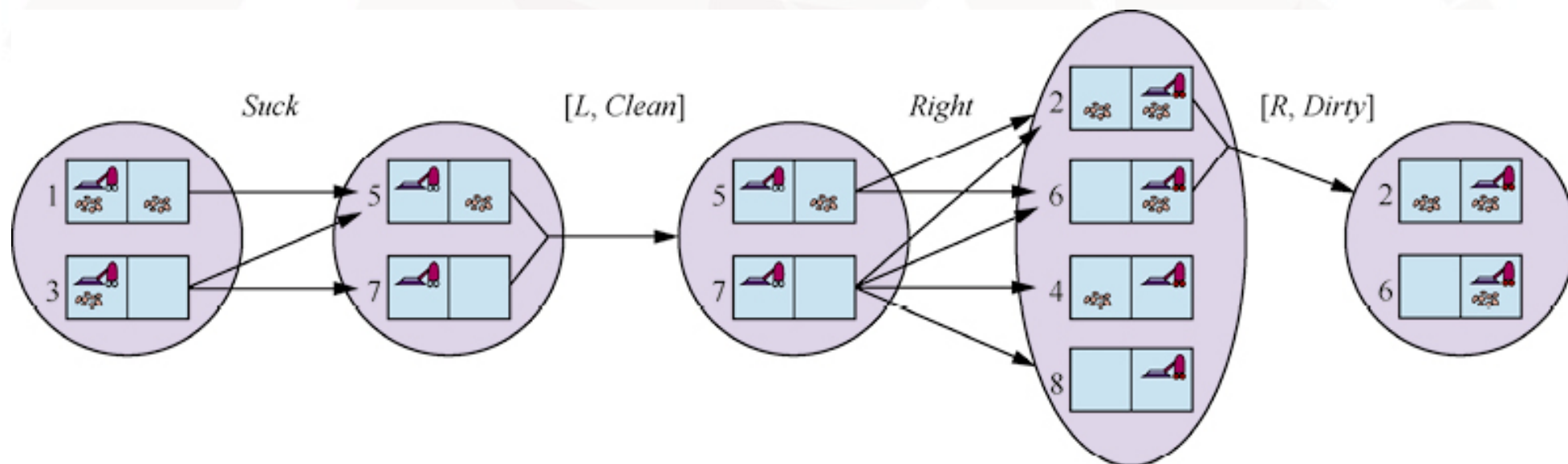
## 例. 部分可观测环境中的吸尘器世界问题的搜索树

- 假设初始的感知[L, Dirty]
- 解: [Suck, Right, **if** State={6} **then** Suck **else** []]



## 例. 部分可观测环境中的幼儿园吸尘器世界

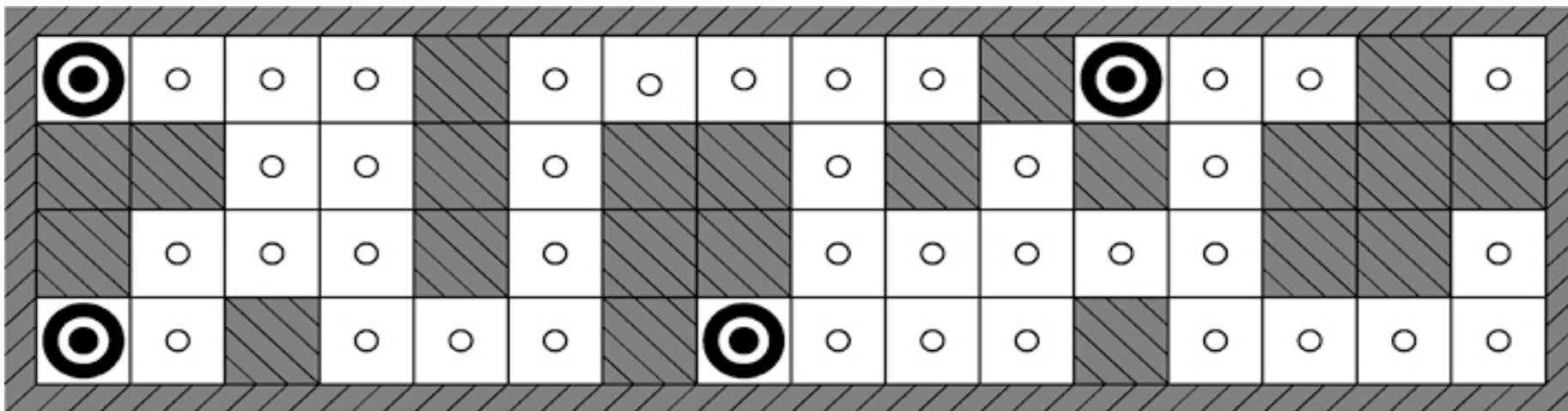
- 任一方格在任一时刻都有可能变脏，除非智能体恰好在那一时刻主动清理该方格



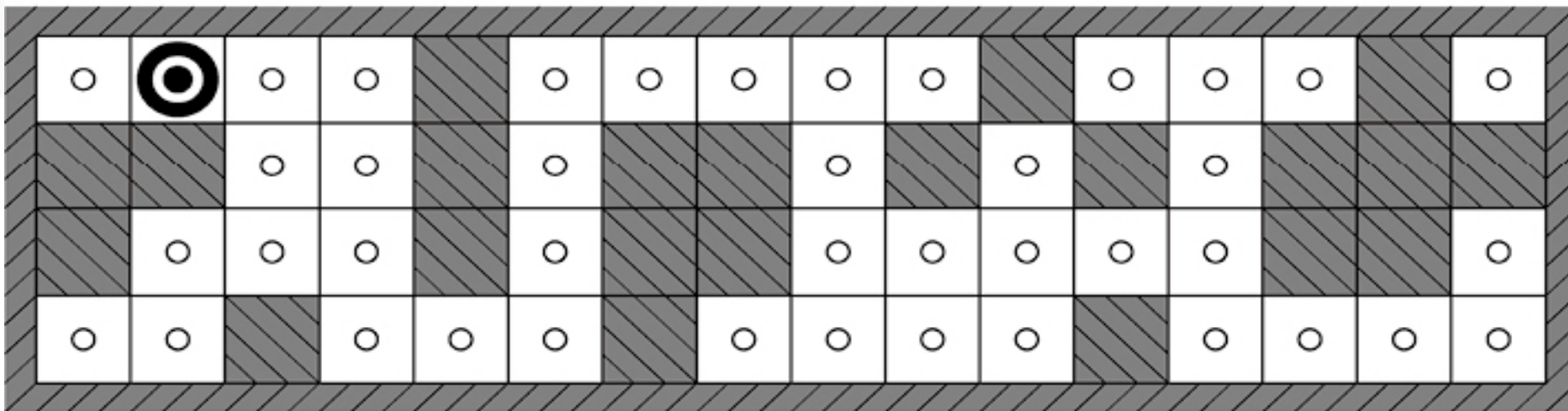


## 例. 机器人定位问题

- 在给定世界地图和一系列感知及行动的情况下，找到自己的位置
- 机器人处在迷宫环境中，可以向四个方向行走
- 机器人配备四个声呐传感器，可以判断在四个方向是否存在障碍物（边界和阻挡）
- 机器人拥有正确的环境地图，但导航失灵（非确定性动作），希望确定自身当前位置
- 感知表示为位向量：1011表示 北、南、西 有障碍物



(a) 观测到 $E_1=1011$ 后, 机器人的可能位置



(b) 观测到 $E_1=1011$ 和 $E_2=1010$ 后, 机器人的可能位置



### 例. 机器人定位问题的搜索过程

- 初始信念状态 $b$ 包含所有位置
- 机器人接收到感知信息1011, 意味着北、南、西方向有障碍物, 利用 $b_0 = \text{Update}(b, 1011)$ 更新信念状态, 找出4个位置如图 (a)
- 机器人继续执行Move动作, 新的信念状态 $b_a = \text{Predict}(b_0, \text{Move})$ , 当接受到感知信息1010后, 利用 $\text{Update}(b_a, 1010)$ 更新信念状态, 找到唯一位置如图 (b)



## 离线搜索

- 计算+行动
- 行动前已经计算出一个完整的解

## 在线搜索

- 计算和行动交替执行
- 先执行一个动作，然后观测环境并计算下一个动作
- 适用于动态、半动态、非确定性环境





## 在线搜索问题

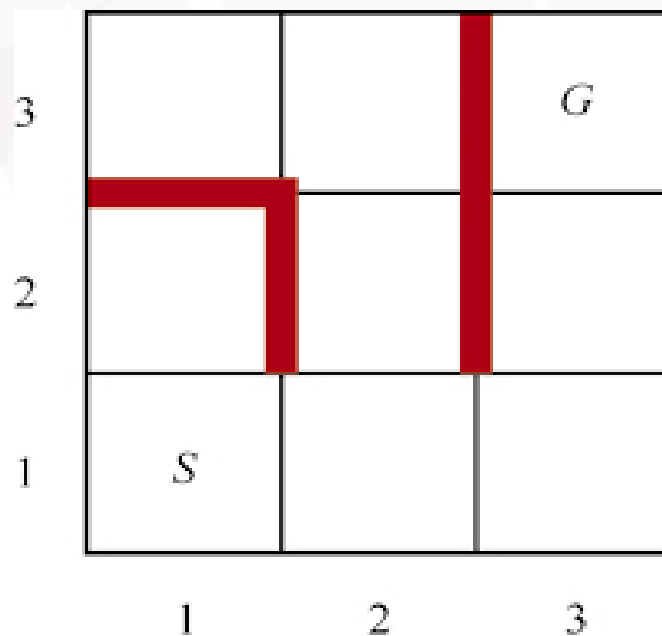
- 假设：环境是确定性的、完全可观测的
- 智能体知道以下内容：
  - $Action(s)$ ：状态 $s$ 下的合法动作
  - $c(s, a, s')$ ：动作代价
  - $Is-Goal(s)$ ：目标测试
- 智能体不能确定 $Result(s, a)$ 的值，除非确实在 $s$ 中执行了动作 $a$
- 智能体可能可以访问一个可容许的启发式函数 $h(s)$ ，估计当前状态到目标状态的距离





## 例. 迷宫问题

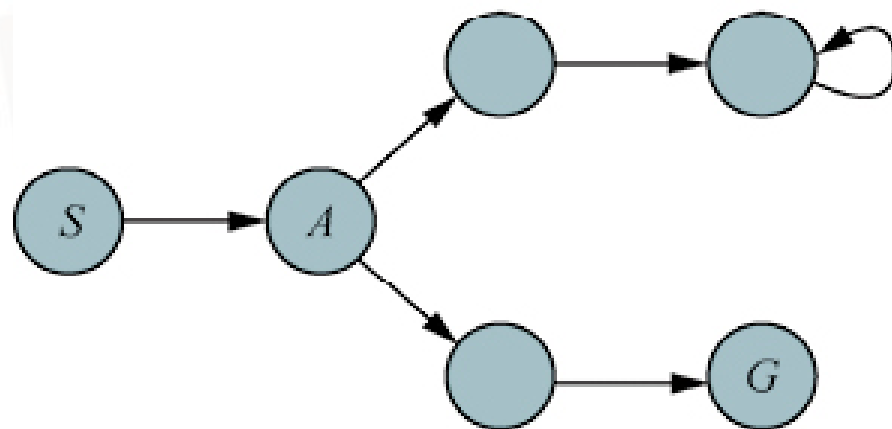
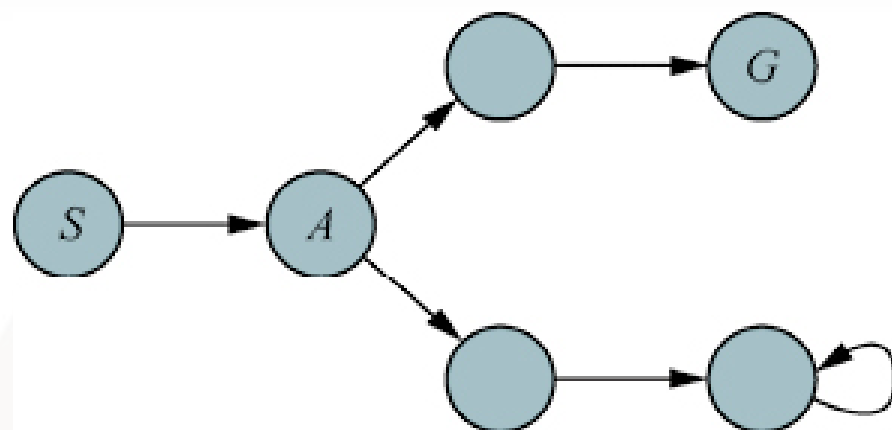
- 智能体不知道从状态(1,1)执行Up动作可以到达状态(1,2)，或者再执行动作Down可以回到状态(1,1)
- 智能体可能知道目标状态(3,3)，启发式函数可以使用Manhattan距离



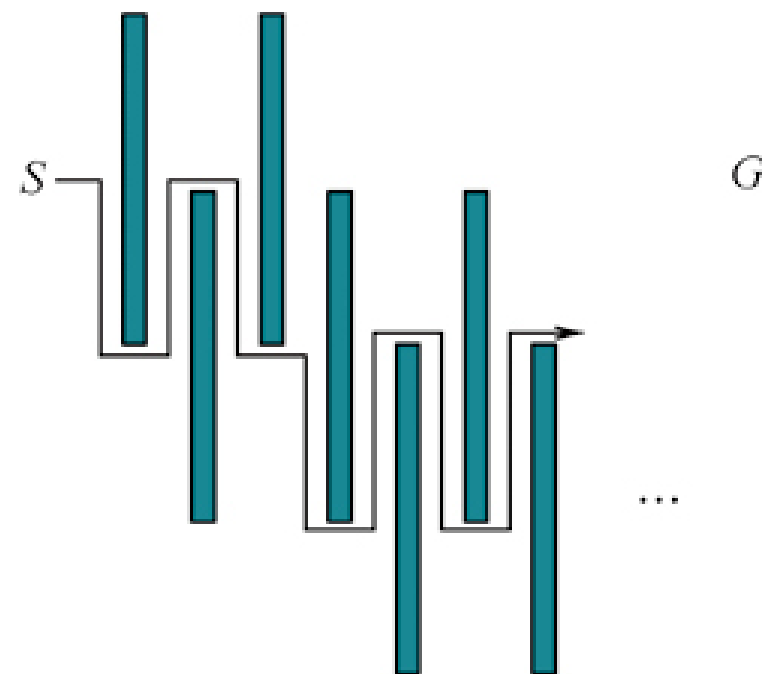


## 竞争比

- 智能体的目标是用最小代价到达目标状态
- 竞争比 = 实际代价 / 最小代价，越小越好
- 在线搜索很容易陷入死胡同（无法到达任何目标的状态），没有一种算法能在所有状态空间中都避免死胡同，任何给定智能体都会在至少一个状态空间中失败
- 某些情况下最好的竞争比是 $\infty$



(a)



(b)



## 在线搜索智能体

- 智能体在每个动作后会收到一个感知，告诉它目前到达了哪一状态，可更新其环境地图
- 更新后的地图用于规划下一步
- 离线算法探索状态空间模型（如A\*），在线算法探索真实世界（实际后继）



**function** ONLINE-DFS-AGENT(*problem*, *s'*) **returns** 一个动作

**persistent:** *s*, *a*, 之前的状态和动作, 初始为空

*result*, 将(*s*, *a*)映射到*s'*的表, 初始为空

*untried*, 将*s*映射到未探索动作列表的表

*unbacktracked*, 将*s*映射到未回溯状态队列的表

**if** *problem*.Is-GOAL(*s'*) **then** **return stop**

**if** *s'*是一个(不在*untried*中的)新状态 **then** *untried*[*s'*]  $\leftarrow$  *problem*.ACTIONS(*s'*)

**if** *s*不空 **then**

*result*[*s*, *a*]  $\leftarrow$  *s'*

将*s*添加到*unbacktracked*[*s'*]的队尾

**if** *untried*[*s'*]为空 **then**

**if** *unbacktracked*[*s'*]为空 **then** **return stop**

*s'*, *a*  $\leftarrow$  null, 使得*result*[*s'*, *b*] = POP(*unbacktracked*[*s'*])的动作*b*

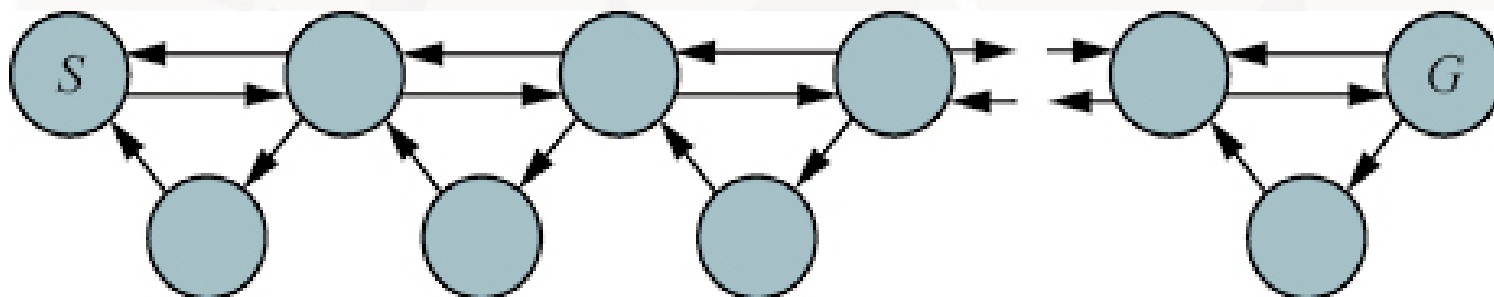
**else** *a*  $\leftarrow$  POP(*untried*[*s'*])

*s*  $\leftarrow$  *s'*

**return** *a*

## 在线局部搜索

- 爬山搜索：陷入局部极大
- 随机重启：智能体无法瞬移到新的初始状态
- 随机游走：过程缓慢





## 增加存储的爬山法

- 存储每个扩展状态 $s$ 到达目标状态的代价的当前最佳估计 $H(s)$
- $H(s)$ 的初始值为 $h(s)$
- 根据智能体在状态空间中获得的经验不断更新 $H(s)$





**function** LRTA\*-AGENT(*problem*, *s'*, *h*) **returns** 一个动作

**persistent:** *s*, *a*, 之前的状态和动作, 初始为空

*result*, 将(*s*, *a*) 映射到*s'*的表, 初始为空

*H*, 将*s*映射到一个代价估计值的表, 初始为空

**if** IS-GOAL(*s'*) **then return** *stop*

**if** *s'*是一个 (不在*H*中的) 新状态 **then**  $H[s'] \leftarrow h(s')$

**if** *s* 不空 **then**

$result[s, a] \leftarrow s'$

$H[s] \leftarrow \min_{b \in ACTIONS(s)} \text{LRTA}^*\text{-COST}(\textit{problem}, s, b, \textit{result}[s, b], H)$

$a \leftarrow \operatorname{argmin}_{b \in ACTIONS(s)} \text{LRTA}^*\text{-COST}(\textit{problem}, s', b, \textit{result}[s', b], H)$

$s \leftarrow s'$

**return** *a*

**function** LRTA\*-COST(*problem*, *s*, *a*, *s'*, *H*) **returns** 一个代价估计值

**if** *s*未定义 **then return**  $h(s)$

**else return**  $\textit{problem}.\text{ACTION-COST}(s, a, s') + H[s]$



## LRTA\* (Learning Real-Time A\*)

- 用 result 表构建了一个环境地图
- 更新刚刚离开的状态的估计代价，然后根据当前的估计代价选择当前最佳动作
- 在状态  $s$  下尚未尝试的动作总是被假定为以最少的可能代价，即  $h(s)$  直接到达目标，这种“不确定性下的乐观主义”鼓励智能体探索新的、可能更有希望的路径

